6502-PICOVDP.

A custom VDP for the AC6502 family, implemented in firmware on PICO9918 PRO v2.0 hardware. No TMS9918 left inside — but the unmodified BIOS still boots.

Status **draft 0.5** | Reference **6502-EMULATOR 3.0.0** | Target **PICO9918 PRO v2.0 / RP2354A**

Firmware-only **no hardware mod** | Ident byte **$AC** · STAT4

Status: draft 0.5. It is implemented in the emulator — `6502-EMULATOR`'s 3.0.0 release, `src/core/IO/Video.ts` — and in firmware, bus and raster included, running on a PICO9918 PRO v2.0 (`6502-PICOVDP`), where its timing is measured (§3, §6, §14, §18). Draft 0.5 adds the built-in font (§7). What changed in each draft is listed under Revision History.

256 COLORS of 4096 · was 15	**2** TILE LAYERS 1, 2, 4 or 8 bpp	**64 KB** VIDEO RAM was 16 KB	**128** REGISTERS was 8	**32** SPRITES / LINE 16 by default · no flicker	**4** CPU PORTS two full port pairs

CONTENTS

1	Goals	9	Display Modes	17	BIOS Impact
2	Target Hardware	10	Sprites	18	Implementation Notes
3	Display Model	11	Palette	19	Omissions & Future Space
4	CPU Interface	12	Compositing	·	Resolved Questions
5	Register Map	13	Scrolling	·	Still Open
6	Status Registers	14	Interrupts	·	Revision History
7	Video RAM	15	Reset State		
8	The Tile Engine	16	Detection		

SECTION 1

Goals

The PICO9918 emulates a TMS9918A faithfully, which the AC6502 does not need. This design keeps the parts of the TMS9918 interface that cost nothing to keep, and spends everything else on capability.

What this design provides

	TMS9918A / PICO9918	6502-PICOVDP
Display modes	Text, Graphics I, Graphics II, Multicolor	**Text, Compact, Graphics, Full — plus a legacy submode**
Tile bit depth	1	**1, 2, 4 or 8, per layer**
Tile layers	1	**2**
Colors on screen	15 + transparent	**256, from 4096**
Palette	fixed	**256 entries, user defined**
Colors per tile	2 (per 8-pixel row at best)	**2, 4, 16 or 256**
Sprites	32 total, 4 per line, 1 color	**64 total, up to 32 per line, up to 255 colors**
Sprite flipping	none	**horizontal and vertical**
Scrolling	none	**hardware, per layer, per pixel**
Interrupts	vblank	**vblank, scanline compare, overflow, collision**
VRAM	16 KB	**64 KB**
VRAM access window	~8 µs between accesses	**unrestricted**
Registers	8	**128**
CPU ports	2	**4**

Design principles

1. **The classic port pair still works.** `$9C00` / `$9C01` behave as they always have. The existing Kernal drives this VDP unmodified (see §17). There is no unlock sequence and no lock: everything new is simply present.
2. **No mode is reachable only by magic.** Registers `$08` – `$7F` are always live.
3. **The 6502 is the bottleneck, not the Pico.** At 1 MHz a `sta VC_DATA` run moves ~250 KB/s at the very most (§4). Every design decision that trades VRAM or CPU writes for VDP-side work is worth taking; every decision that demands more CPU writes is not. This is why VRAM is 64 KB and not 512 KB, and why hardware scrolling matters more than extra tile capacity.

4. **Nothing is at a fixed address.** Every table is placed by a base register. Reset writes two things at fixed addresses, the default palette (§11) and the built-in font (§7), and software may move or overwrite either.

Non-goals

- TMS9918 Graphics II and Multicolor modes. Dropped — see §19 for why Graphics II in particular did not survive. The *capability* Graphics II offered, one color pair per pattern row, is a first-class feature of the tile engine (§8).
- F18A compatibility. The F18A register set is not implemented and not emulated.
- A GPU/coprocessor.
- A blitter or VRAM fill/copy engine. See §19.

SECTION 2

Target Hardware

PICO9918 PRO v2.0 (RP2354A) only.

MCU	RP2354A: an RP2350A, dual Cortex-M33, with 2 MB of flash in the package
SRAM	520 KB
System clock	352 MHz (VGA preset 2), the clock §18's budget is measured at. The stock firmware boots at preset 0, 252 MHz, and preset 1 is 302.4 MHz; this design budgets for neither
Video out	VGA 640×480@60, HDMI, or SCART RGB, via the FFC dongles. The timing in §3 and §14 is the VGA raster's; SCART's interlaced 480i/576i timing is not specified by this revision
Color depth	12 bits (4-4-4 R/G/B on GPIO 2–13)

Why not v1.0–1.3 (RP2040)

The first draft of this reasoning priced a line against one VGA line, 31.78 µs: ~9,600 cycles at 302.4 MHz against an estimated ~7,400 of work on an M33, and ~8,000 at 252 MHz against ~12,000 on an M0+. But a display line is rendered once per **two** VGA lines (§3, §18), 63.56 µs, which doubles every budget: ~19,200 cycles on the M33 and ~16,000 on an RP2040. On that arithmetic the RP2040 no longer fails outright, and the case against it is now the one it was always partly about — a v1.x profile would mean cutting to roughly 16 sprites per line and dropping 8bpp, two renderers and a capability query in every piece of software. The M33's side has since been measured (§18): at 352 MHz it takes both cores to build the worst-case line in time, where the estimate had one core doing it with half the line to spare. The target stays the PRO v2.0. An RP2040 was not measured, and would need to be before a v1.x profile was considered.

If a v1.x profile is wanted later, `STAT6` (§6) already reports capability bits for exactly this purpose.

No hardware modification required

The AC6502 already wires the PICO9918's `MODE` pin (GPIO 28) to A0 and `MODE1` (GPIO 29) to A1. The existing `tmsWrite` PIO program already samples both: `in pins, 16` from GPIO 14 reaches GPIO 29, and MODE1 lands in bit 31 of the FIFO word, so the write side needs only its handler changed.

The read side needs more. `tmsRead` answers a read from a word staged ahead of it — pin directions, the data port's prefetched byte and the status byte, 28 of its 32 bits already spoken for — and chooses between two outputs on `MODE`. Four ports need `MODE1` sampled as well (`in pins, 1` becomes `in pins, 2`) and a second port's data and status staged beside the first's, which is a rewrite of the read program and its handler rather than a one-word change. It is still all firmware.

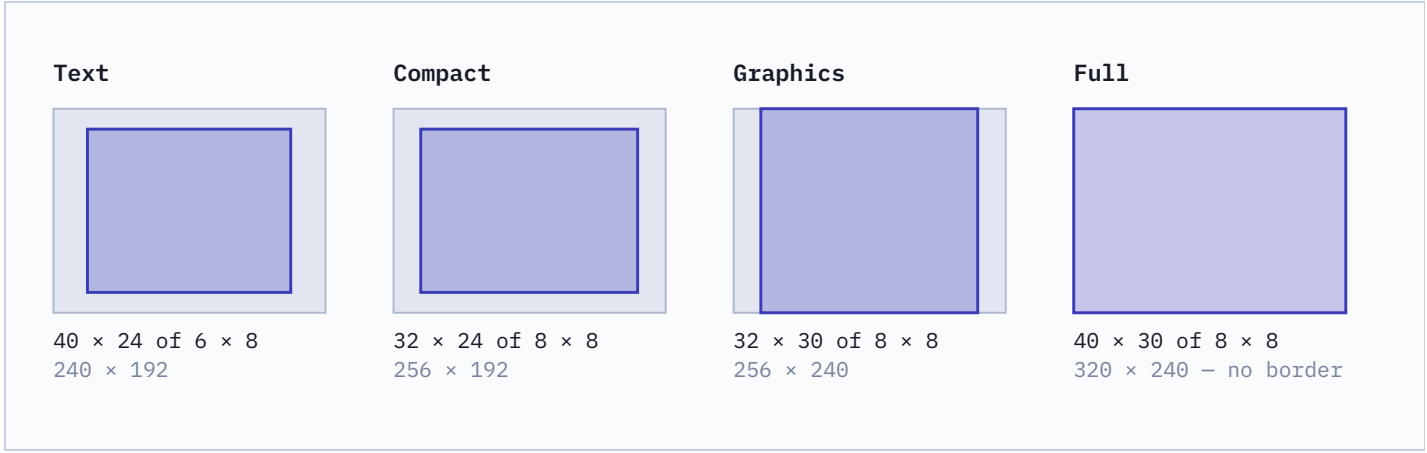
> **This is a firmware-only project.**

SECTION 3

Display Model

The PICO9918 renders a **320 × 240 virtual frame** at one byte (palette index) per pixel, and expands it 2× in both axes through a 256-entry RGB lookup into the 640 × 480 VGA raster. This VDP keeps that pipeline exactly.

Virtual frame	320 × 240, 8-bit palette index per pixel
Physical output	640 × 480 @ 59.94 Hz (2× scale)
Pixel clock	25.175 MHz
VGA line period	31.78 µs, 525 lines a frame
Display line	Two VGA lines, 63.56 µs — one line of the 240-line virtual frame
Frame period	16.68 ms
VGA vertical blanking	45 VGA lines (22.5 display lines) ≈ 1.43 ms ≈ 1,430 CPU cycles @ 1 MHz
Palette	256 entries × 12-bit RGB (4096 colors)



Active picture (outlined) inside the 320 × 240 virtual frame (grey), drawn to scale.
Full mode is the only one with no border at all — and the only one whose tables outgrow a single 1 KB block.

Active areas within the 320 × 240 frame:

MODE	CELLS	PIXELS	POSITION	BORDER
Text	40 × 24 of 6 × 8	240 × 192	x 40–279, y 24–215	40 px left/right, 24 lines top/bottom
Compact	32 × 24 of 8 × 8	256 × 192	x 32–287, y 24–215	32 px left/right, 24 lines top/bottom
Graphics	32 × 30 of 8 × 8	256 × 240	x 32–287, y 0–239	32 px left/right, none top/bottom
Full	40 × 30 of 8 × 8	320 × 240	the whole frame	none

On a 4:3 monitor at 640 × 480 the pixels are square, so Full mode's 8 × 8 cells are square on screen and the frame has no border at all. Text mode's 6 × 8 cells are taller than they are wide, as they are on a TMS9918.

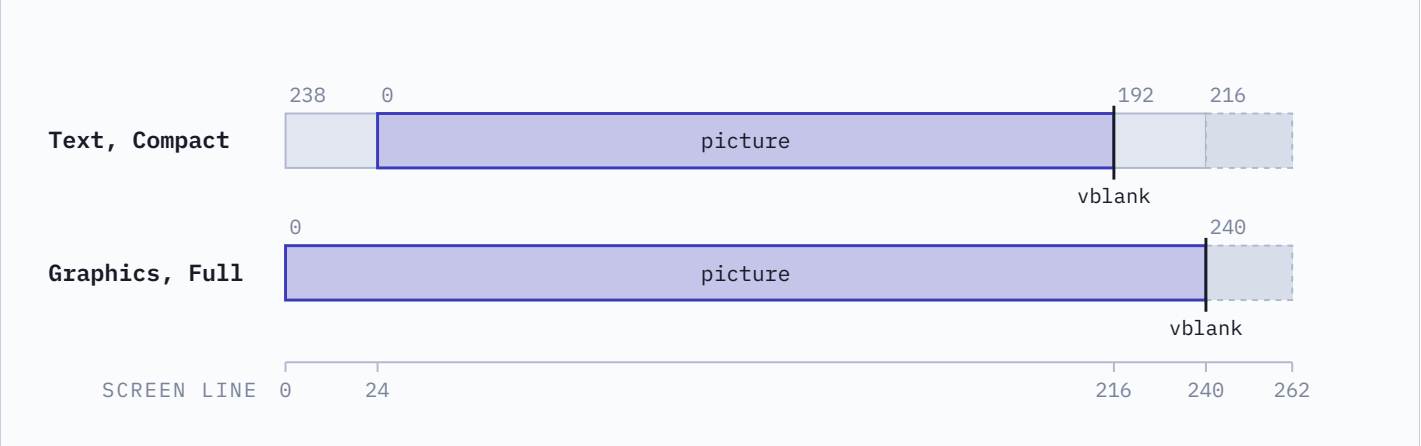
Text mode therefore occupies exactly the same physical screen area it does today (240 virtual pixels → 480 VGA pixels wide, either way). Graphics mode is the same width as today's 256-pixel modes but 240 lines tall instead of 192.

The raster does not change with the mode. A frame is 525 VGA lines: 262 **screen lines** of two VGA lines each, numbered 0–261 from the top of the frame, and one odd VGA line after screen line 261 that begins no line of its own. Screen lines 0–239 are the 240 rows of the frame above; screen lines 240–261 and the odd VGA line are the VGA raster's vertical blanking, in every mode.

Scanline numbering used by `IRQLINE` and `STAT2` is the **display line**, which counts from the first line of the active picture *in the current mode* — not from the top of the frame. Display line 0 is the first visible line, the count runs up through the picture, the bottom border, vertical blanking and the top border, and wraps after 261. It is the screen line less the picture's top border:

```
display line = (screen line − top border) mod 262
top border   = 24 in Text and Compact, 0 in Graphics and Full
```

In the 192-line modes, then, display line 0 is screen line 24, the top border's screen lines 0–23 are display lines 238–261, scanned before the picture they sit above, and the odd VGA line follows display line 237.



Screen lines along the bottom are the raster, and the same in every mode; the numbers above each bar are its display lines. **A frame begins at screen line 0**, and vertical blank fires where the picture ends — screen line 216 or 240. Grey is border, dashed is VGA blanking; the odd VGA line follows screen line 261.

MODE	ACTIVE	PICTURE OCCUPIES	OUTSIDE THE PICTURE
Text, Compact	192 lines	display lines 0–191, screen lines 24–215	bottom border 192–215; VGA blanking 216–237 and the odd VGA line; top border 238–261
Graphics, Full	240 lines	display lines 0–239, screen lines 0–239	VGA blanking 240–261 and the odd VGA line

Counting from the picture rather than from the frame means a raster split stays put across a mode change: `IRQLINE = 80` is ten character rows down whatever mode is running. It is also how the TMS9918 counts, which matters for §14.

When a line is built

Display line N is built from the registers, VRAM and palette as they stand when display line N − 1 begins. That is when the PICO9918 structure asks its render core for the line, which then has the whole of line N − 1 to draw it (§18). Line 0 is built as line 261 begins. A line not finished in that time is shown as §18's *Late lines* describes.

So a write the CPU makes while line N is being scanned shows from line N + 2 — the picture, the backdrop in the border beside it (§11) and the border lines outside the picture alike. A handler for `IRQLINE` = N, which runs while line N is scanned (§14), changes the picture from line N + 2: set `IRQLINE` two lines above the first line that should change.

A change of display mode takes effect from the next line built, so a frame in which the mode changes may be torn. Between Text and Compact, or between Graphics and Full, that is all it does. Between a 192-line and a 240-line geometry it also moves the top border, and the screen lines stay where they are, so the display line numbering moves

instead: the next line to begin is numbered 24 more than it would have been (192 lines to 240) or 24 less (240 lines to 192). The display lines skipped match no `IRQLINE`, the ones repeated match it again, and vertical blank still fires exactly once in the frame (§14).

Display off (`MODE1` b6 clear) draws the backdrop over the picture area. Neither layer is drawn and no sprite is evaluated, so no overflow or collision is reported; the display line counter, `STAT0` b7 and the interrupts carry on.

SECTION 4

CPU Interface

Four byte-wide ports in I/O slot 8, decoded from A1:A0. Slot 8 mirrors them across `$9C00`–`$9FFF`; use the canonical addresses.

A1	A0	ADDRESS	NAME	WRITE	READ
0	0	$9C00	VC_DATA	VRAM data, port A	VRAM data, port A
0	1	$9C01	VC_REG / VC_STATUS	command, port A	status, port A
1	0	$9C02	VC_DATA2	VRAM data, port B	VRAM data, port B
1	1	$9C03	VC_REG2 / VC_STATUS2	command, port B	status, port B

Two independent ports

`$9C02` / `$9C03` are a second, complete copy of the interface. Each port pair has its own:

- VRAM address pointer
- read/write direction latch
- read-ahead prefetch byte
- first-byte/second-byte flip-flop
- status register selector (`STATSEL_A` / `STATSEL_B`)

Both ports address the same 64 KB of VRAM and write the same register file — which includes `VBANK` and `VINC`. Those two are not per port.

This exists for three reasons:

1. **Interrupt safety.** The hazard the AC6502 documentation warns about — an interrupt landing between the two halves of a command pair — disappears if interrupt handlers use port B and foreground code uses port A, and a handler that writes `VBANK` or `VINC` puts them back. No `sei` / `cli` around VDP work. Status reads are split for the same reason (§6): a handler acknowledges through `STAT1` without clearing the `STAT0` flags foreground code polls.
2. **VRAM to VRAM copies without a RAM buffer.** `lda VC_DATA2 : sta VC_DATA` in an unrolled loop is 8 cycles per byte, both pointers advancing.

3. **Two live cursors.** Park port B on the sprite attribute table and port A on the name table and neither needs re-addressing.

Command protocol

Identical to the TMS9918 on both ports. Two writes to the command port:

```
1st write:  payload byte P
2nd write:  command byte C
```

C	EFFECT
`%1rrrrrrr`	Write register `r` (0–127) with value `P`
`%01aaaaaa`	Set VRAM pointer for **write** to `{VBANK[1:0], a[5:0], P[7:0]}`
`%00aaaaaa`	Set VRAM pointer for **read** to the same, and prefetch

The TMS9918 decodes 3 register bits and the F18A 6; this VDP decodes 7. Legacy writes of `$80` – `$87` reach registers 0–7 unchanged.

The payload is held in a latch of its own. A register write does not change either port's pointer or prefetch byte.

Any access to a pair's data port, and any read of its status port, resets that pair's flip-flop — so a program that loses track of it recovers by reading status once.

VRAM pointer

The pointer is a full 16-bit counter. The command protocol sets bits 13:0; bits 15:14 come from `VBANK` (§5), sampled when the address command completes. After each `VC_DATA` access the pointer advances by the signed stride in `VINC` (reset value `+1`), **carrying into the bank bits** — a streaming write runs off the end of a bank into the next one — and wrapping from `$FFFF` to `$0000`, or from `$0000` to `$FFFF` on a negative stride.

The carry lives in that port's pointer and nowhere else. `VBANK` is never changed by it, and a `VBANK` write does not move a pointer already set: it applies to the next address command, on either port.

> This is the one place where TMS9918 behavior is deliberately broken: a real TMS9918 wraps within 16 KB. Nothing in the AC6502 software suite relies on that wrap.

Reads are prefetched. This is TMS9918 behavior and the reason the read-then-advance pattern works. Exactly:

OPERATION	EFFECT, IN ORDER
Set a read address	Fetch the byte at the pointer into the prefetch; advance

OPERATION	EFFECT, IN ORDER
Set a write address	Nothing more
Read `VC_DATA`	Return the prefetch; fetch the byte at the pointer into it; advance
Write `VC_DATA`	Store at the pointer; load the written byte into the prefetch; advance

The direction latch records which kind of address was set last and nothing reads it: either data-port operation works whichever way a port was pointed.

Access timing

There is no minimum interval between VRAM accesses. VRAM is RP2350 SRAM, not a DRAM array being time-shared with the raster, so the ~8 µs gap a real TMS9918 demands does not exist. The 65C02's fastest back-to-back port access is `sta abs` at 4 cycles — 4 µs at 1 MHz, 2 µs at 2 MHz — which leaves the RP2350 roughly 700 cycles at 352 MHz to service each access. The firmware's bus interrupt takes about 150, and nothing on the card holds an access off for longer than about 300 more, so accesses 2 µs apart — a 2 MHz 6502's back-to-back `lda VC_DATA`, or a read straight after the address command that sets it — are right every time, measured under the heaviest load the card can be given (§18).

One operation does not fit that budget. A `FONT` command (§5) copies 2 KB into VRAM, far more work than an access may take, so the card does not copy as the write arrives. The write records the load, and the load completes at the next vertical blank (§7, §14). The copy itself follows that line start a 64-byte chunk at a time, with accesses served in between, and an access that reaches a chunk not yet copied waits for that chunk alone, a fraction of a microsecond: every access after the line start finds the font whole (§18).

Sustained throughput through an unrolled `sta VC_DATA` run — the ceiling:

CPU	BYTES/S	960-BYTE TABLE	8 KB PATTERN SET	FULL 64 KB
1 MHz	~250 K	3.8 ms	33 ms	262 ms
2 MHz	~500 K	1.9 ms	16 ms	131 ms

A real loop is slower. `sta VC_DATA : iny : bne` is 9 cycles a byte, ~111 KB/s at 1 MHz; copying from memory with `lda (zp),y` is about 14, ~71 KB/s. The upload times in §8 and §19 are at the ceiling — multiply by 2.2 to 3.5 for a loop.

Writing during active display is permitted and will not corrupt the raster, but changing a structure the current frame is still drawing will tear. Structures are sampled line by line as §3 describes; the palette cache takes a write at once (§11), and the next line built uses it.

SECTION 5

Register Map

128 registers, all write-only. Read back state through the status registers (§6).

Registers `$02` – `$06` are **aliases** of registers in the layer and sprite blocks — the same storage under two addresses — so that legacy register writes and the symmetric new layout describe the same hardware.

$00–$07 LEGACY CORE			
$00	**MODE0**	$00	**b1 M3** legacy mode select (§9); b0 EXTVID *(ignored)*; b7:2 reserved
$01	**MODE1**	$00	b7 *(ignored, 4/16K)*; **b6 DISP** display enable; **b5 IE** vblank IRQ enable; **b4 M1** and **b3 M2** legacy mode select (§9); b2 reserved; **b1 SPRSIZE** 0 = 8×8, 1 = 16×16; **b0 SPRMAG** magnify ×2
$02	**L0NAME**	$00	Layer 0 name table base, × `$400` . Alias of `$10` .
$03	**L0ATTR**	$00	Layer 0 attribute table base, × `$400` . Alias of `$11` .
$04	**L0PAT**	$00	Layer 0 pattern table base, × `$800` . Alias of `$12` .
$05	**SPRATTR**	$00	Sprite attribute table base, × `$80` . Alias of `$20` .
$06	**SPRPAT**	$00	Sprite pattern table base, × `$800` . Alias of `$21` .
$07	**COLOR**	$00	b7:4 default foreground index, b3:0 backdrop/border index. The foreground indexes the 16-entry palette group of the layer drawing it, named by its `LxPAL` (§8); the backdrop is in `L0PAL` 's (§11)

`M1` , `M2` and `M3` select the display mode only while `VMODE` (register `$0D`) is `$0` , which it is at reset. See §9 for what each combination resolves to and what `VMODE` does instead.

Base register fields are widened from the TMS9918's. `L0NAME` was 4 bits, now 8 (× `$400` → reaches `$3FC00` , of which bits 5:0 are meaningful in 64 KB). `L0PAT` / `SPRPAT` were 3 bits, now 8 (× `$800` → `$F800` within 64 KB). `SPRATTR` stays × `$80` over 8 bits, reaching `$7F80` . Legacy values land exactly where they used to, for any value whose bits a TMS9918 ignores are zero. Every table address wraps at 64 KB (§7).

$08–$0F ACCESS AND INTERRUPTS			
$08	**VBANK**	$00	VRAM address bits A21:A14, sampled by an address command (§4). Bits 1:0 implemented (64 KB); the rest are stored, ignored, and reserved for a larger VRAM.
$09	**VINC**	$01	VRAM auto-increment stride, signed 8-bit, −128…+127. `$00` = no increment.

$0A	IRQEN	$00	b0 vblank, b1 scanline compare, b2 sprite overflow, b3 sprite collision; b7:4 ignored
$0B	IRQLINE	$00	Scanline compare value, display line 0–255
$0C	PALBASE	$3F	Palette base, b5:0 × $400; b7:6 ignored. Reset = $FC00.
$0D	VMODE	$00	b3:0 display mode (§9); b7:4 reserved, ignored
$0E	STATSEL_B	$00	b3:0: which status register port B returns; b7:4 ignored
$0F	STATSEL_A	$00	b3:0: which status register port A returns; b7:4 ignored

STATSEL_A is at $0F because the F18A puts its status select at R15; anyone carrying F18A habits lands in the right place. The two selectors are separate so an interrupt handler reading status on port B cannot disturb foreground code reading status on port A.

$10–$17 LAYER 0			
$10	L0NAME	$00	Name table base, × $400 *(= $02)*
$11	L0ATTR	$00	Attribute table base, × $400 *(= $03)*
$12	L0PAT	$00	Pattern table base, × $800 *(= $04)*
$13	L0SCRX	$00	Horizontal scroll, pixels — bits 7:0; bit 8 is L0CTRL b6
$14	L0SCRY	$00	Vertical scroll, pixels
$15	L0CTRL	$3C	b1:0 bit depth; b3:2 attribute source; b4 layer enable; b5 index 0 opaque; b6 horizontal scroll bit 8; b7 reserved — see §8
$16	L0PAL	$00	b3:0 palette group high bits — see §8; b7:4 ignored
$17	—	—	Reserved

Identical layout, different reset values.

$18–$1F LAYER 1			
$18	L1NAME	$00	Name table base, × $400
$19	L1ATTR	$00	Attribute table base, × $400
$1A	L1PAT	$00	Pattern table base, × $800
$1B	L1SCRX	$00	Horizontal scroll, pixels — bits 7:0; bit 8 is L1CTRL b6
$1C	L1SCRY	$00	Vertical scroll, pixels

$1D	**L1CTRL**	$0C	Same bits as `L0CTRL`. Reset: layer disabled, index 0 transparent.
$1E	**L1PAL**	$00	b3:0 palette group high bits; b7:4 ignored
$1F	—	—	Reserved

`L0CTRL` resets to `$3C` — 1bpp, no attribute table, enabled, index 0 opaque — so that a bare text screen in `VMODE` Text has a solid background colored by `COLOR`. `L1CTRL` resets to `$0C` — the same, but disabled and with index 0 transparent, which is what an overlay layer wants.

While `VMODE` = `$0` the bit-depth and attribute-source fields of `L0CTRL` are ignored and derived from the legacy mode bits instead, and so is b5: index 0 is transparent there, as TMS9918 color 0 is (§9).

$20–$27 SPRITES			
$20	**SPRATTR**	$00	Attribute table base, × $80 *(= $05)*
$21	**SPRPAT**	$00	Pattern table base, × $800 *(= $06)*
$22	**SPRCOUNT**	$20	Active sprite slots, 0–64; larger values act as 64. Slots at or above this index are not evaluated.
$23	**SPRCTRL**	$27	b0 sprites enable; b1 collision detection enable; b2 `$D0` terminates the sprite list; b3 detailed collision reporting; b5:4 bit depth (00 = 1, 01 = 2, 10 = 4, 11 = 8); b7:6 reserved
$24	**SPRLIMIT**	$10	Maximum sprites drawn per scanline, 1–32; larger values act as 32. 0 draws no sprites and reports an overflow on every line a sprite covers.
$25	**SPRPAL**	$00	b3:0 palette group high bits for sprites — `LxPAL`'s equivalent; b7:4 ignored
$26–$27	—	—	Reserved

Sprite size and magnification live in `MODE1` b1:b0, as on the TMS9918. They are not duplicated here. While `VMODE` = `$0` the bit-depth field is ignored, b2 acts as set, `SPRPAL` is ignored, and sprites render with TMS9918 semantics (§9).

`SPRCTRL` resets to `$27` — enabled, collision on, `$D0` terminator active, detailed collision off, 4bpp.

`SPRLIMIT` resets to 16: the most sprites the firmware draws on the worst-case line with a quarter of the line to spare (§18). Any value up to 32 works — the worst case at 32 still builds in time, with less in hand, and a line that ever does not is shown as §18's *Late lines* says. It is also a way to reproduce a low per-line limit for period-correct flicker, should anyone want it.

$28–$7F FONT, AND RESERVED			
$28–$2F	—	—	Reserved for a blitter (§19)

$30	**FONT**	$00	Load a built-in font (§7). **b7** destination: 0 = layer 0's pattern table, at `L0PAT` × `$800`; 1 = layer 1's, at `L1PAT` × `$800`. **b6:0** font ID: `$00` = CP437, 6 × 8; `$01`–`$7F` reserved. Every write is a command
$31–$7F	—	—	Reserved

Write 0 to a reserved register. `$31`–`$7F` are reserved for a second sprite bank, raster effect tables or additional layers (§19). A reserved register — `$17`, `$1F`, `$26`–`$27`, `$28`–`$2F` or `$31`–`$7F` — stores what is written and has no effect.

`FONT` stores what is written as well, and a write naming a reserved font ID does nothing more. Reset sets it to `$00` without loading anything through it; reset installs font `$00` itself (§15).

SECTION 6

Status Registers

Reading a status port returns the register named by b3:0 of that port's `STATSEL`. Sixteen are defined.

#	NAME	CONTENTS
0	STAT0	b7 **F** — a picture has ended, set **regardless of `IRQEN`**; b6 **OVF** — a sprite has been dropped; b5 **COL** — two sprites have collided; b4:0 low five bits of the first sprite dropped, 0 while OVF is clear. All four since `STAT0` was last read
1	STAT1	b0 vblank, b1 scanline compare, b2 overflow, b3 collision — the latched sources that are still **enabled** in `IRQEN`; b7:4 reserved
2	STAT2	Current display line, low 8 bits. Lines 256–261 alias to 0–5; `STAT3` b0 disambiguates.
3	STAT3	b0 vertical blanking: the display line is at or past the current mode's active line count, borders included; b1 horizontal blanking, during either VGA line of the display line (see below); b7:2 reserved
4	STAT4	**`$AC`** — identification byte
5	STAT5	Firmware version, BCD: high nibble major, low nibble minor
6	STAT6	Capabilities: b0 two layers, b1 8bpp layer, b2 sprite flip, b3 hardware scroll, b4 scanline IRQ, b5 64 KB VRAM, b6 reserved (for a blitter, §19), b7 built-in font: register `$30` and font `$00`, loaded at reset (§7). This card reads `$BF`
7	STAT7	Full index (0–63) of the first sprite dropped on the most recent overflowing line since `STAT0` was last read; 0 if none
8–15	STAT8–STAT15	Collision bitmap — bit *s* mod 8 of `STAT(8 + s/8)`, *s*/8 rounded down, is set if sprite *s* has collided since `STAT0` was last read. Only maintained while `SPRCTRL` b3 is set.

`STAT0` keeps the TMS9918's shape exactly, including the five-bit sprite field, so that code testing `bit VC_STATUS` / `bmi` for vblank still works.

b7 is a flag, not an interrupt. It is the TMS9918's F bit: it sets when the active picture ends, whether or not `IRQEN` b0 is on, because `IRQEN` governs the `/INT` pin and nothing else. Polling `STAT0` for vertical blank with interrupts disabled is a common

idiom and it has to work. The other interrupt sources do not appear in b7 at all — read `STAT1` for those.

The flags are sticky. Nothing clears `F`, `OVF`, `COL`, the index field, `STAT7` or the collision map but a read of `STAT0` or a reset (§15), however many frames go by — as on the TMS9918, so a program that looks once a second still sees the collision that happened in between.

Acknowledgement. The two reads clear different things, and that is what makes port B safe for an interrupt handler:

- **Reading `STAT0`** clears `F`, `OVF`, `COL` and the index field, `STAT7` and the collision map — and the `STAT1` latches for vertical blank, overflow and collision, the interrupts those flags stand for. A TMS9918 handler that reads status to acknowledge its interrupt still does.
- **Reading `STAT1`** clears every `STAT1` latch, and nothing else.

A read clears only what it returned. A status byte is staged before the read that returns it (below), so a flag or latch that sets in between is not in the byte; the read leaves it set, and the next read shows it.

So a handler on port B reads `STAT1`, and foreground code polling `STAT0` on port A still finds `F` set. Read `STAT7` and the collision map before `STAT0`, which clears them.

`STAT3` b1 is advisory. A status read is served from a byte staged before the read arrives (§2), so it can lag the raster; nothing finer than a display line should be timed from it. Measured on the PRO: at each line start the byte is restaged, and `/INT` driven to match, 1.35 µs later on average and 3.2 µs at most, so `STAT2`, `STAT3` b0 and the flags can be trusted to the line. b1 follows each VGA line's horizontal blanking a few tenths of a microsecond late (§18).

Reading any status register resets that port's command flip-flop.

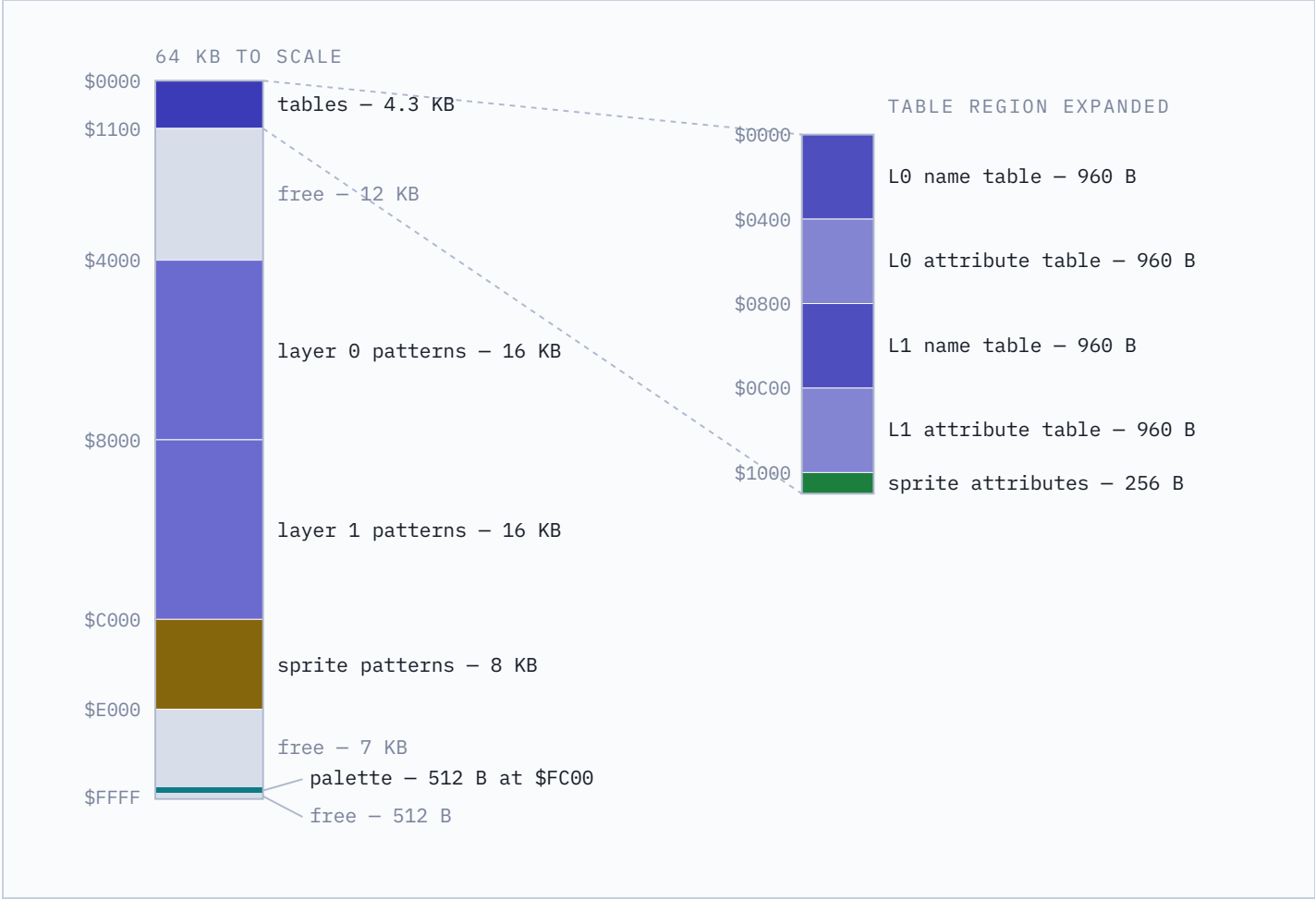
SECTION 7

Video RAM

64 KB, flat, addressed as a 16-bit space. No banking is visible to software beyond `VBANK` supplying the top two address bits.

64 KB is four times the TMS9918 and takes a 1 MHz 65C02 a quarter of a second to fill end to end. More would be memory the CPU cannot realistically populate. `VBANK` is specified as a full byte so a later revision can grow to 4 MB without changing the command protocol.

Every structure is placed by a base register. This is the **recommended** layout for graphics mode, not a requirement:



4.3 KB of tables govern 40 KB of pattern data. Every base register is free to move these anywhere; only the alignment granularity is fixed. The 960-byte tables each sit in their own 1 KB block, leaving 64 bytes of each block unused.

```
$0000 - $03BF   Layer 0 name table           960 B    L0NAME = $00
$0400 - $07BF   Layer 0 attribute table      960 B    L0ATTR = $01
$0800 - $0BBF   Layer 1 name table           960 B    L1NAME = $02
$0C00 - $0FBF   Layer 1 attribute table      960 B    L1ATTR = $03
$1000 - $10FF   Sprite attribute table       256 B    SPRATTR = $20
$1100 - $3FFF   free                          12 KB
$4000 - $7FFF   Layer 0 pattern table         16 KB   L0PAT = $08
$8000 - $BFFF   Layer 1 pattern table         16 KB   L1PAT = $10
$C000 - $DFFF   Sprite pattern table           8 KB   SPRPAT = $18
$E000 - $FBFF   free                           7 KB
$FC00 - $FDFF   Palette                      512 B    PALBASE = $3F
$FE00 - $FFFF   free                         512 B
```

Text mode after reset keeps today's layout, which the existing Kernal already programs:

```
$0000 - $03BF   Layer 0 name table           960 B    L0NAME = $00
$0800 - $0FFF   Layer 0 pattern table          2 KB   L0PAT = $01
$FC00 - $FDFF   Palette                      512 B
```

Reset has already put the built-in font at `$0800` (below), so once software selects Text and sets `L0PAT` = `$01` the pattern table is in place.

Alignment rules:

STRUCTURE	BASE REGISTER GRANULARITY	SIZE
Name table	1 KB	768, 960 or 1200 B, by geometry
Attribute table	1 KB (×`$40` for layer 0 in the legacy submode, §9)	per cell: as the name table; per pattern group: 32 B; per pattern row: 2 KB
Pattern table	2 KB	2 KB (text), up to 16 KB (graphics)
Sprite attribute table	128 B	256 B
Sprite pattern table	2 KB	8 KB for 256 patterns at 4bpp, 16 KB at 8bpp
Palette	1 KB	512 B

The name table is 960 bytes in **both** Text and Graphics — 40 × 24 and 32 × 30 are the same count — so a swap between them needs no reallocation, and Graphics I's 768 bytes fit in the same 1 KB block. Full mode is the exception at 1200 bytes, which spans two 1 KB blocks; budget 2 KB for each of its tables. The attribute table, when per-cell, always has exactly the same geometry as the name table it belongs to. A per-pattern-row table is 2 KB, so it does not fit the 1 KB slot the layout above gives an attribute table.

Every table address wraps at 64 KB: a table whose base plus offset passes `$FFFF` continues at `$0000`.

The built-in font

The card holds a font of its own. Reset puts it in VRAM, and `FONT` (register `$30`, §5) puts it into either layer's pattern table on command.

Font `$00`	CP437, 6 × 8
Size	2,048 bytes: 256 glyphs of 8 rows, glyph *n* at offset *n* × 8
Row byte	b7 is the leftmost pixel. Bits 7:2 are the 6-pixel Text cell, and bits 1:0 are always 0
SHA-256	`b2adc19efd10870196bad05d84eae51500599935c80f13d608a4f62278260577`

The bytes are the character set BIOS 1.x keeps in AC6502 ROM at `$B800` and uploads to `$0800` (6502-BIOS v1.6, `Chars.asm`). `6502-PICOVDP` holds them as `fonts/cp437-6x8.bin`. **The hash is normative.** The font is 1bpp, so it works in every geometry: in the 8 × 8 ones a glyph sits in the left six pixels of its cell.



Font `$00`, every glyph drawn from its 2,048 bytes: row = high nibble of the character code, column = low nibble. **No glyph uses bits 1:0**, so each fits the Text cell's six pixels exactly.

At reset. Reset writes font `$00` to `$0800`–`$0FFF`, after the default palette (§11), at power-on and at every `RST` (§15). Both are in place before the card serves its first access after reset. The address is fixed. It is not `L0PAT` × `$800`, which reset leaves at `$0000`, but where the Text layout above and the Kernal put the pattern table. Reset values do not change: software still selects Text and sets `L0PAT` = `$01`, and finds the font already there.

On command. A write to `FONT` through either port's command protocol (§4) loads a font:

1. **Every write is a command,** even a write of the value the register already holds. The value is stored, as any register's is.
2. **The destination is taken at the write.** b7 names layer 0's pattern table, at `L0PAT` × `$800`, or layer 1's, at `L1PAT` × `$800`, and the base is sampled as the write arrives, as an address command samples `VBANK` (§4). A later write to `L0PAT` or `L1PAT` does not move a pending load. The highest base, `$F800`, ends at `$FFFF`, so a load never wraps.
3. **A reserved font ID** is stored and does nothing else. It does not cancel a pending load.

4. **One load is pending per destination.** A second write for the same destination before the first completes replaces it. Loads for both layers can be pending together.
5. **A pending load completes at the line start where vertical blank fires** (§14): as part of that line start, before `STAT0` F sets, before `/INT` is asserted for it, and before the registers and VRAM are taken for the line it begins building (§3). Once software has seen F, or a vertical blank interrupt, that followed the write, the load is complete, which is within a frame. The copy lands in blanking, and no picture line is built from a half-copied table.
6. **Until it completes,** the destination keeps what it held. Reads return the old contents, and a write there is overwritten when the load lands. Read `STAT0` to clear F, write `FONT`, and wait for F before touching that range.
7. **In every other respect the copy is a VRAM write.** Where it overlaps the palette window, the palette cache takes it (§11). It moves no port's pointer or prefetch byte, as no register write does (§4).
8. **A reset cancels every pending load** (§15).

`STAT6` b7 says the card has all of this (§6, §16).

SECTION 8

The Tile Engine

Both layers, in every mode, are the same engine: a **name table** mapping screen cells to pattern indices, a **pattern table** of pixel data, and a **source of color** for each cell. Mode selection (§9) chooses the geometry; `LxCTRL` chooses everything else, per layer.

Bit depth — `LxCTRL` b1:b0

B1:B0	DEPTH	BYTES PER 8×8 TILE	COLORS PER CELL	TILES	FULL SET	UPLOAD @ 1 MHZ
00	1bpp	8	2, independent fg and bg	256	2 KB	8 ms
01	2bpp	16	4	512	8 KB	33 ms
10	4bpp	32	16	512	16 KB	66 ms
11	8bpp	64	256	256	16 KB	66 ms

The pattern index is the name table byte, plus attribute b7 as a ninth bit at 2 and 4bpp when there is an attribute byte (§8's color byte, below) — which is why those two depths reach 512 tiles and 1bpp and 8bpp reach 256. A tile's pattern starts at `LxPAT × $800 + index × bytes per tile`. Upload times are at §4's ceiling.

Patterns are stored row by row from the top, pixels left to right, the most significant bit or nibble leftmost — 1, 2, 4 or 8 bytes per row.

In text mode the cell is 6 pixels wide, at every depth: the leftmost 6 pixels of each row are drawn and the rest ignored. At 1bpp that is the top 6 bits of one byte, which is the TMS9918 text format exactly, the format of the character set in AC6502 ROM at `$B800`, and the format of the built-in font (§7). A horizontally flipped Text cell mirrors the six pixels drawn, not all eight.

The low depths are not a consolation prize. VRAM is not the binding constraint on this machine — **upload time is** — and a 1bpp tile set costs a quarter of what a 4bpp one costs to get into the VDP. A full 1bpp set, 256 tiles, is 2 KB and 8 ms at 1 MHz; the same 256 tiles at 4bpp are 8 KB and 33 ms, two frames' worth. 8bpp is the odd one out: it costs the most VRAM but is the *cheapest to render*, because there is no unpacking to do.

Attribute source — `LxCTRL` b3:b2

Where the color byte for a cell comes from:

B3:B2	SOURCE	ADDRESS OF THE COLOR BYTE	BYTES
00	Per cell	`LxATTR + cell`	768, 960 or 1200
01	Per pattern group	`LxATTR + (pattern >> 3)`	32
10	Per pattern row	`LxATTR + (pattern × 8) + row`	2 KB
11	None	*no fetch* — see below	0

Per cell is the general case, and the only one in which two cells sharing a pattern can have different colors. **Per pattern group** is the TMS9918's Graphics I color table: one byte per eight consecutive patterns, 32 bytes for the whole screen. **Per pattern row** is Graphics II's scheme: one color byte per pattern row, so an 8×8 pattern carries eight fg/bg pairs down its height.

Sources 01 and 10 are meant for 1bpp. At 2, 4 and 8bpp they still fetch a byte from the same address and use it as an attribute byte: per pattern group indexes by the 8-bit name byte, so the ninth bit of the byte it finds moves all eight names in the group; per pattern row gives each pixel row of a cell its own attribute byte, looked up by the row before any vertical flip is applied.

Per-pattern-group is what makes Graphics I fall out of the engine rather than needing a mode of its own. Per-pattern-row is the one genuinely useful thing Graphics II could do that nothing else could — eight color pairs down the height of a pattern — kept here as a feature of the engine even though legacy Graphics II itself is not supported (§19).

None means no attribute fetch at all. At 1bpp the whole layer is colored by `COLOR` (register `$07`) — which is what today's text mode does, and why the existing Kernal needs no changes. At 2bpp the layer uses sub-palette 0 of the quarter `LxPAL` selects; at 4bpp it draws from palette row `LxPAL`; at 8bpp the value is the palette index. In every case there is no flipping, no priority and no ninth pattern-index bit.

The color byte

At 1bpp it is a pair of nibbles, foreground in b7:4 and background in b3:0, each indexing the 16-entry group selected by `LxPAL`:

```
b7 b6 b5 b4 | b3 b2 b1 b0
foreground  | background
```

This is the TMS9918 convention unchanged, which is why `VideoSetColor` still means what it means and a Graphics I color table still colors what it colored.

At 2, 4 and 8bpp it is an attribute byte:

BIT	2BPP AND 4BPP	8BPP
b3:0	sub-palette	*ignored*
b4	flip horizontally	flip horizontally
b5	flip vertically	flip vertically
b6	priority — §12 level 4 on layer 0, 6 on layer 1: in front of ordinary sprites, and a layer 0 tile in front of an ordinary layer 1 tile too	same
b7	pattern index bit 8 → 512 tiles	*ignored*

Palette mapping

A **palette group** is `2^bpp` consecutive palette entries. The group number is `LxPAL × 16 + subpal`, and the palette index of a pixel is `(group × 2^bpp + value) & $FF`. In practice:

DEPTH	GROUP SIZE	GROUPS AVAILABLE	PALETTE INDEX OF A PIXEL
1bpp	—	—	`LxPAL × 16 + nibble` — fg or bg, as a 4-bit index
2bpp	4	64	`(LxPAL & 3) × 64 + subpal × 4 + value`
4bpp	16	16	`subpal × 16 + value`; with no attribute byte, `LxPAL × 16 + value`
8bpp	256	1	`value`

At 4bpp the sixteen groups already cover the whole palette, so the formula leaves nothing of `LxPAL` whenever an attribute byte carries a sub-palette. With attribute source "none" there is no such byte, and `LxPAL` is the sub-palette: the layer draws from palette row `LxPAL`. At 2bpp its low two bits pick which quarter of the palette the sixty-four groups are drawn from, so all 256 entries stay reachable. At 1bpp it picks which sixteen colors the fg/bg nibbles name.

Transparency

A pixel whose value is 0 within its group is transparent, unless `LxCTRL` b5 (index 0 opaque) is set. At 1bpp this applies to **both** nibbles — a foreground nibble of 0 is as transparent as a background nibble of 0, exactly as TMS9918 color 0 is. In the legacy submode layer 0's b5 is ignored and its index 0 is always transparent (§9); layer 1's b5 still applies.

Setting index 0 opaque gives a layer the full `2^bpp` colors and makes it occlude everything behind it. Layer 0 resets that way — though the legacy submode it resets into ignores the bit (§9) — and layer 1 does not.

SECTION 9

Display Modes

`VMODE` (register `$0D`) b3:0 selects the geometry. Everything else about a layer comes from `LxCTRL` (§8).

VMODE	MODE	CELLS	CELL	PIXELS	NAME TABLE	POSITION IN THE FRAME
$0	**Legacy**	from M1/M2/M3				see below
$1	**Text**	40 × 24	6 × 8	240 × 192	960 B	x 40–279, y 24–215
$2	**Compact**	32 × 24	8 × 8	256 × 192	768 B	x 32–287, y 24–215
$3	**Graphics**	32 × 30	8 × 8	256 × 240	960 B	x 32–287, y 0–239
$4	**Full**	40 × 30	8 × 8	320 × 240	1200 B	the whole frame
$5–$F	reserved — behaves as $0					

Full mode fills the 320 × 240 frame edge to edge — no border, square cells on a 4:3 monitor. It is the mode for anything that wants the whole screen: a title page, a game field with no letterboxing, or a 40 × 30 text display six rows taller than Text mode and with square characters, by running it at 1bpp with per-cell attributes.

It costs three small things and nothing else. Its name and attribute tables are 1200 bytes rather than 960, so each occupies two 1 KB blocks instead of one. Its horizontal scroll needs nine bits (§13). And 320 pixels of layer per line is 25% more layer work than 256, which makes it the most expensive mode in the design: its worst-case line is the one §18's budget is measured against.

The names describe geometry and nothing else. **Compact** is the TMS9918's Graphics I grid — which is why a legacy Graphics I program lands there — but it is not "Graphics I": new software can run that 32 × 24 grid at 4bpp with per-cell attributes and flipping, and can equally run **Graphics** at 1bpp with a 32-byte color table if that is what the art wants. Geometry and coloring are orthogonal, and only the legacy submode pins particular combinations.

Compact exists for software that wants the 192-line height and its top and bottom borders, or simply wants a smaller name table and a cheaper line. Graphics is the one most new software should reach for.

`VMODE` `$1` and `$3` both use a 960-byte name table, so a mode change between them needs no reallocation.

The legacy submode

`VMODE` = `$0`, the reset value, hands mode selection to the TMS9918's `M1`, `M2` and `M3` bits — `MODE1` b4, `MODE1` b3 and `MODE0` b1 — and pins layer 0's bit depth and attribute source to match. `L0CTRL`'s depth and attribute fields are ignored while this is in effect, and so is its opacity bit: index 0 is always transparent, as TMS9918 color 0 is, whatever `L0CTRL` b5 holds. Its enable bit still applies. So do `L0SCRX`, `L0SCRY`, `L0CTRL` b6 and `L0PAL`, which a legacy program never writes, and layer 1 is unaffected. A reserved `VMODE` value, `$5`–`$F`, behaves exactly as `$0`.

M1	M2	M3	TMS9918 MODE	GEOMETRY	L0 DEPTH	L0 ATTRIBUTE SOURCE
1	×	×	Text	40 × 24	1bpp	none — `COLOR` colors the screen; no sprites
0	0	0	Graphics I	32 × 24	1bpp	per pattern group, base = `L0ATTR` × `$40`
0	0	1	Graphics II	32 × 24	1bpp	*not supported* — falls back to Graphics I
0	1	×	Multicolor	32 × 24	1bpp	*not supported* — falls back to Graphics I

One register reinterpretation applies, and only here: **`L0ATTR` is scaled by `$40`, not `$400`**, so that a Graphics I program's 32-byte color table lands where it wrote it.

Graphics II and Multicolor are not implemented. A program selecting either gets Graphics I geometry and coloring and will draw the wrong thing — but it will draw *something* rather than hanging the raster. There is no capability bit for them; the detection probe in §16 is the signal, since a program that finds this VDP rather than a TMS9918 already knows it should be using `VMODE` instead. §19 sets out why Graphics II was cut.

Sprites in the legacy submode take TMS9918 semantics, whatever `SPRCTRL` and `SPRPAL` hold:

- 1 bit per pixel, 8 bytes per 8 × 8 pattern; a 16 × 16 sprite with index N draws patterns N to N + 3 as its quadrants, from `SPRPAT × $800 + N × 8`
- a sprite's first row is drawn on display line **Y + 1**, and Y of `$E1`–`$FF` means −31… −1, so `$FF` puts the first row on line 0
- attribute b3:0 is a direct index into palette row 0; color 0 is invisible, and still collides
- attribute b7 is the early-clock bit, shifting the sprite 32 pixels left rather than adding 256 to X; b4–b6 are ignored
- a slot whose Y is `$D0` always ends the list

- in Text mode there are no sprites: none is evaluated, drawn, counted or collided

`SPRCTRL` b0, b1 and b3, `SPRCOUNT` and `SPRLIMIT` still apply. At their reset values a legacy program sees up to 16 sprites on a line where a TMS9918 showed four.

What this buys

Text mode and Graphics I both run exactly as they do today — but for the per-line sprite limit — which means the unmodified BIOS boots (§17) and `graphics-1.asm` still draws what it drew. Neither cost the engine anything: Text is a geometry plus "no attribute fetch", and Graphics I is the Compact geometry plus one attribute-source value. The compatibility is a consequence of the design rather than a layer bolted onto it, which is why it is worth having — and why Graphics II, which *would* have been a bolted-on layer, is not here.

Recommended starting point for new software

```
lda #$03                    ; Graphics: 32x30 of 8x8
sta VC_REG
lda #($80 | $0D)            ; register $0D = VMODE
sta VC_REG

lda #%00110010              ; 4bpp, per-cell attributes, enabled, index 0 opaque
sta VC_REG
lda #($80 | $15)            ; register $15 = L0CTRL
sta VC_REG

lda #$01                    ; attribute table at $0400, clear of the name table (§7)
sta VC_REG
lda #($80 | $11)            ; register $11 = L0ATTR
sta VC_REG
```

Then upload the tables, place or disable the sprites — `SPRCTRL` b0, or clear b2 and set `SPRCOUNT`, since `$D0` is on a 240-line screen (§10) — and turn the display on with `MODE1` b6.

SECTION 10

Sprites

64 sprite slots, up to 32 drawn per scanline, at 1, 2, 4 or 8 bits per pixel.

Attribute table — 4 bytes per slot, up to 64 slots = 256 bytes. Only slots below `SPRCOUNT` are evaluated.

OFFSET	CONTENTS
+0	**Y** — top edge, as a display line (§3). 0–239 position the sprite down the picture; 241–255 mean −15…−1, entering from the top; 240 is the first row below a 240-line picture, and in the 192-line modes 192–240 are all below it. The legacy submode reads this byte as the TMS9918 did (§9).

OFFSET	CONTENTS
+1	**X** — left edge, bits 7:0
+2	**Pattern** — index into the sprite pattern table
+3	**Attributes**

Attribute byte:

BIT	MEANING
b3:0	Sub-palette, 0–15
b4	Flip horizontally
b5	Flip vertically
b6	Priority — 0: behind layer 1, 1: in front of layer 1
b7	X bit 8 — see below

Horizontal position. X is a 9-bit value: bits 7:0 from attribute offset +1, bit 8 from attribute b7. Values **0–383** are coordinates, covering the widest mode with room to spare; values **384–511** mean −128…−1, which is how a sprite enters from the left. One rule in every `VMODE` mode — −128 is four times the width of a magnified 16 × 16 sprite — and the early-clock bit in the legacy submode (§9).

X and Y are relative to the picture, not the frame: X = 0 is the picture's left edge, x 32 of the frame in Compact. Sprites are clipped to the picture and never drawn in the border.

Ending the list. `SPRCOUNT` always bounds the table, which makes evaluation a fixed cost. In addition, while `SPRCTRL` b2 is set — the reset state, and forced in the legacy submode — a slot whose Y is `$D0` ends the list there, as on the TMS9918.

Y = `$D0` is row 208, which is off the bottom of a 192-line legacy screen but **on** a 240-line Graphics screen. Software using the full height should clear `SPRCTRL` b2 and rely on `SPRCOUNT`; software that wants the familiar idiom can leave it alone. Keeping both costs one bit and one comparison.

Patterns — depth from `SPRCTRL` b5:4, value 0 always transparent.

SIZE	1BPP	2BPP	4BPP	8BPP	LAYOUT
8 × 8	8 B	16 B	32 B	64 B	one row at a time, MSB/high nibble leftmost
16 × 16	32 B	64 B	128 B	256 B	four 8 × 8 patterns — the sprite's index N and N + 1, N + 2, N + 3 — as quadrants in TMS9918 order: top-left, bottom-left, top-right, bottom-right

The pattern index counts 8 × 8 patterns, whatever the sprite size: pattern N starts at `SPRPAT × $800 + N × B`, where B is the 8 × 8 size at the depth — 8, 16, 32 or 64 bytes. That is the TMS9918's rule at every depth, and it is why §7's 8 KB holds 256 patterns at 4bpp.

Sprite palette mapping follows §8's formula with `SPRPAL` in `LxPAL`'s place: the palette index of a sprite pixel is `((SPRPAL × 16 + subpal) × 2^bpp + value) & $FF`. At 1bpp that is `((SPRPAL × 16 + subpal) × 2 + 1) & $FF` — the formula, not §8's 1bpp nibble row. The legacy submode has its own rule (§9).

Size is global, from `MODE1` b1. `MODE1` b0 magnifies every sprite ×2, giving 16 × 16 or 32 × 32 on screen. Flipping applies to the whole sprite, quadrant arrangement included.

Per-line limit. Sprites are evaluated in table order, on the lines of the picture only. When more than `SPRLIMIT` sprites cover a line, the excess — highest indices first — is dropped for that line only. A sprite covers a line if its rows do, even when it lies wholly off the picture to the left or right, so it counts toward the limit. `STAT0` b6 is set; `STAT0` b4:0 holds the first sprite dropped since `STAT0` was last read, and `STAT7` the first dropped on the most recent overflowing line. Sprites do not flicker; if you want flicker you must implement it yourself.

Priority among sprites. Lower table index wins, as on the TMS9918, and it is settled before the layers are: the lowest-index sprite with a non-transparent pixel owns that pixel, and only then does its attribute b6 set the level it competes with the layers at (§12). A higher-index sprite does not show through a lower-index one, even where the lower one loses to a layer.

Collision. When `SPRCTRL` b1 is set — the reset state — any two non-transparent sprite pixels landing on the same pixel of the picture set `STAT0` b5 and, if enabled, raise an interrupt, once a frame (§14). This is the TMS9918's flag, with the TMS9918's limitation: it says *something* collided, not what. Sprites dropped by the per-line limit do not collide.

Detailed collision. Setting `SPRCTRL` b3 additionally records **which** sprites were involved, as a 64-bit map across `STAT8` – `STAT15`. Both members of every colliding pair are marked. The map accumulates until `STAT0` is read, with the `COL` bit it details (§6). b3 does nothing while b1 is clear.

This is opt-in because it is the one collision feature with a real cost: working out which sprites every collision involved is roughly 2,500 extra cycles on a worst-case line (§18). Leave b3 clear and you pay nothing; the plain sticky bit needs no owner tracking at all.

Collision is tested before priority resolution, so a sprite hidden behind a layer still collides.

SECTION 11

Palette

512 bytes at `PALBASE`, 256 entries of 12-bit RGB.

```
entry n + 0:  %0000RRRR
entry n + 1:  %GGGGBBBB
```

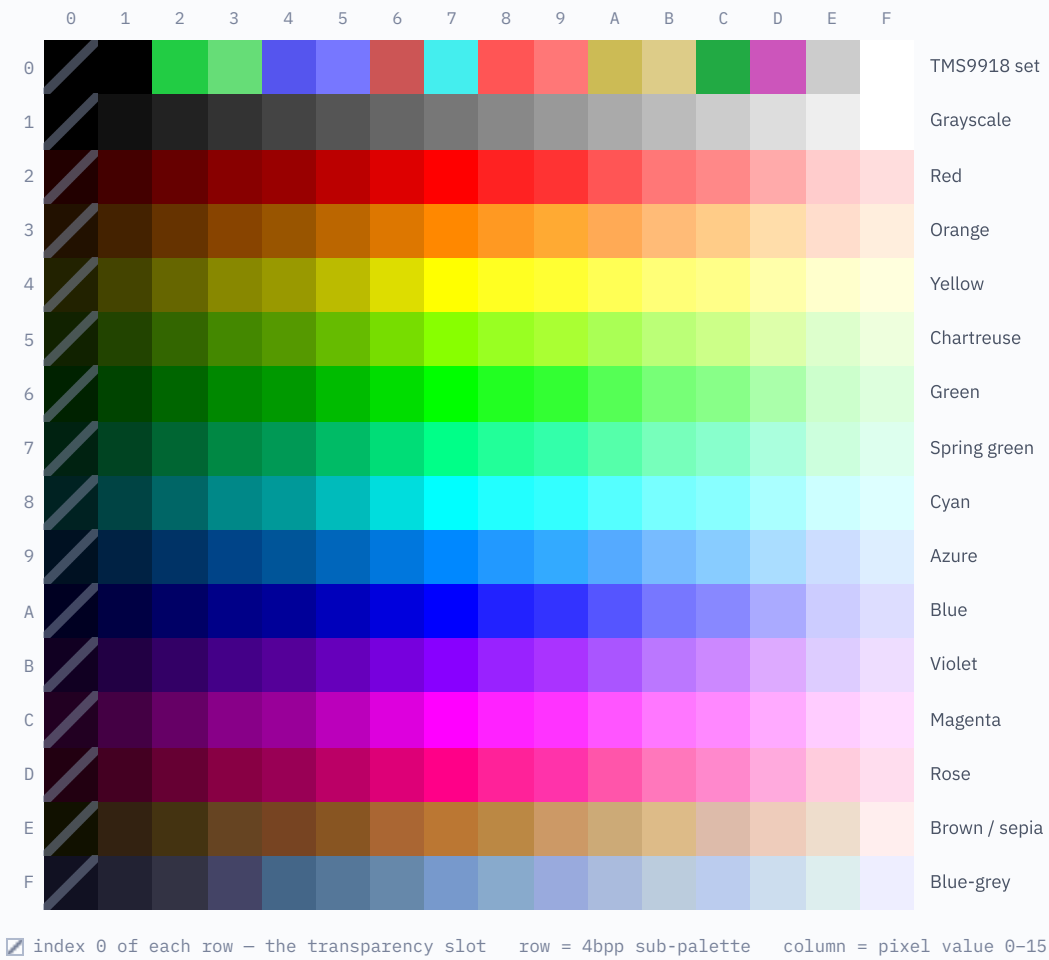
The high nibble of the first byte is ignored.

How the palette is divided into groups depends on bit depth — §8 has the rule and the table. At 4bpp it is 16 groups of 16; at 2bpp, 64 groups of 4; at 8bpp the division does not apply. In every case the entry at value 0 within a group is the transparency slot: no layer or sprite drawing at that group's depth uses its stored color, except a layer with `LxCTRL` b5 set. Entries are shared across depths, though — one depth's transparency slot is an ordinary color at another — and the backdrop, `(L0PAL × 16) + (COLOR & $0F)`, can be any entry.

The backdrop — the color behind every layer and the border outside the active area — is palette entry `(L0PAL × 16) + (COLOR & $0F)`, taken for each display line as §3 builds it, border included.

Writes take effect immediately. The VDP holds a cache of the palette as expanded RGB pairs and snoops VRAM writes that fall inside the 512-byte window at `PALBASE`, updating the cache on the spot; the next line built uses it (§3). There is no dirty flag to set and no reload command. Moving `PALBASE` re-reads the whole window.

Default palette. Reset writes a default palette into VRAM at `$FC00` and loads the cache from it. It is organized as sixteen rows of sixteen — so that a 4bpp sub-palette selector picks a row — after the Commander X16's arrangement:



The full 256-entry default palette, every swatch rendered from the spec's 12-bit values. **Row 0 is the sixteen TMS9918 colors the ACE shows today**, so the same nibble means the same color it did before; rows 2–15 are generated from a formula rather than hand-picked.

ROW	CONTENTS
0	The sixteen TMS9918 colors the ACE shows today, index 0 transparent
1	Grayscale ramp, `$000` to `$FFF`
2–13	Twelve hues at roughly 30° intervals — red, orange, yellow, chartreuse, green, spring green, cyan, azure, blue, violet, magenta, rose
14	Brown / sepia
15	Blue-grey

Each hue row is a sixteen-step ramp, dark at index 0, the pure hue at index 7, tinted toward white at index 15 — so a 4bpp tile that selects one row gets a complete shading ramp for that hue, and picking a row is picking a color scheme.

Values, as 12-bit `$RGB`:

row	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	000	000	2C4	6D7	55E	77F	C55	4EE	F55	F77	CB5	DC8	2A4	C5B	CCC	FFF
1	000	111	222	333	444	555	666	777	888	999	AAA	BBB	CCC	DDD	EEE	FFF
2	200	400	600	800	900	B00	D00	F00	F22	F33	F55	F77	F88	FAA	FCC	FDD
3	210	420	630	840	950	B60	D70	F80	F92	FA3	FA5	FB7	FC8	FDA	FDC	FED
4	220	440	660	880	990	BB0	DD0	FF0	FF2	FF3	FF5	FF7	FF8	FFA	FFC	FFD
5	120	240	360	480	590	6B0	7D0	8F0	9F2	AF3	AF5	BF7	CF8	DFA	DFC	EFD
6	020	040	060	080	090	0B0	0D0	0F0	2F2	3F3	5F5	7F7	8F8	AFA	CFC	DFD
7	021	042	063	084	095	0B6	0D7	0F8	2F9	3FA	5FA	7FB	8FC	AFD	CFD	DFE
8	022	044	066	088	099	0BB	0DD	0FF	2FF	3FF	5FF	7FF	8FF	AFF	CFF	DFF
9	012	024	036	048	059	06B	07D	08F	29F	3AF	5AF	7BF	8CF	ADF	CDF	DEF
A	002	004	006	008	009	00B	00D	00F	22F	33F	55F	77F	88F	AAF	CCF	DDF
B	102	204	306	408	509	60B	70D	80F	92F	A3F	A5F	B7F	C8F	DAF	DCF	EDF
C	202	404	606	808	909	B0B	D0D	F0F	F2F	F3F	F5F	F7F	F8F	FAF	FCF	FDF
D	201	402	603	804	905	B06	D07	F08	F29	F3A	F5A	F7B	F8C	FAD	FCD	FDE
E	110	321	431	642	742	852	A63	B73	B84	C96	CA7	DB8	DBA	ECB	EDC	FEE
F	112	223	334	446	468	579	68A	79C	8AC	9AD	ABD	BCD	BCE	CDE	DEE	EEF

Row 0 is taken from the values the AC6502 documentation publishes for the TMS9918's sixteen colors, quantized to 4 bits per channel — so the same nibble means the same color it does today, and `COLOR = $1F` is black on white with no software change.

The table is normative. Rows 2–15 were generated, not hand-picked, so they can be regenerated at a different depth or ramp shape. The generator takes each row's index 7 entry as its base — the twelve hues, and `$B73` and `$79C` for the two neutral rows:

```
def ramp(base):                         # base = pure hue, 4 bits per channel
    out = []
    for n in range(16):
        if n <= 7:  out.append(tuple(round(b * (n + 1) / 8) for b in base))
        else:       out.append(tuple(b + round((15 - b) * (n - 7) / 9) for b in base))
    return out
```

The `/ 9` in the upper half is deliberate: `/ 8` would end every ramp at pure white and waste fourteen entries on the same color.

`round` there is Python's, which breaks a tie toward the even number. C's `round()` breaks it away from zero and gives `$335`, `$456` and `$68B` for three entries of row F where the table has `$334`, `$446` and `$68A`. Transcribe the table rather than regenerating it.

> Reset clobbers `$FC00`–`$FDFF`, and `$0800`–`$0FFF` with the built-in font (§7, §15). Nothing in the reset-time memory map lives at `$FC00`, and `$0800` is where the Text layout keeps its font anyway.

SECTION 12

Compositing and Priority

Each pixel gathers candidates from the backdrop, both layers and the sprites. Each candidate carries a priority level; the highest non-transparent candidate wins.

FRONT

LEVEL		
6	**Layer 1** tile with attribute b6 set	PRIORITY BIT
5	**Sprite** with attribute b6 set	PRIORITY BIT
4	**Layer 0** tile with attribute b6 set	PRIORITY BIT
3	**Layer 1** tile, normal	DEFAULT
2	**Sprite** with attribute b6 clear	DEFAULT
1	**Layer 0** tile, normal	DEFAULT
0	**Backdrop**	DEFAULT

BACK

Seven priority levels from one bit per tile and per sprite. The four marked **default** are the arrangement with no priority bits set anywhere: backdrop, layer 0, sprites, layer 1.

The default arrangement — no priority bits set anywhere — is backdrop, layer 0, sprites, layer 1, back to front. Setting a layer 0 tile's priority bit lifts that tile above ordinary sprites, which is how a sprite walks behind scenery — and above an ordinary layer 1 tile, level 4 over level 3. Setting a sprite's priority bit lifts it above layer 1, which is how a cursor or a health bar stays on top of everything.

Only a cell with an attribute byte has a priority bit: a 1bpp layer, or one whose attribute source is "none", is always at level 1 or 3 (§8). Legacy sprites ignore b6 and are always at level 2 (§9). The sprites resolve among themselves first, by table index, and the winning sprite's b6 is what competes here (§10).

A disabled layer contributes nothing. Outside the legacy submode's layer 0 (§9), a layer whose `LxCTRL` b5 (index 0 opaque) is set never contributes a transparent pixel, so it occludes everything below it.

Collision detection considers only sprite-against-sprite overlap, before priority resolution. A sprite hidden behind a layer still collides.

SECTION 13

Scrolling

`LxSCRX` and `LxSCRY` offset each layer's view into its map, in pixels, independently, with wrapping. Scrolling costs nothing per frame — it is two register writes.

MODE	MAP SIZE	X WRAPS AT	Y WRAPS AT
Text	240 × 192	240	192
Compact	256 × 192	256	192
Graphics	256 × 240	256	240
Full	320 × 240	320	240

`LxSCRX` is 9 bits — the register plus `LxCTRL` b6 — because Full mode is 320 pixels wide and an 8-bit register could only reach 255 of them. The other modes leave b6 clear, but it is still added if set. Exactly, for a pixel at picture column x on display line y, with map width W and height H:

```
mapX = (x + ((LxCTRL.b6 × 256 + LxSCRX) mod W)) mod W
mapY = (y + (LxSCRY mod H)) mod H
```

So increasing `LxSCRX` moves the picture left and increasing `LxSCRY` moves it up. The cell, its attribute and its pattern pixel all come from the map position.

The map is the same size as the screen, so the map wraps around onto itself. To scroll through content larger than one screen, write the incoming row or column into the cells the scroll is about to expose — the standard technique, and at one row of 32 or 40 bytes per 8 pixels of travel it is well within a 1 MHz budget.

The most immediate use is text. The Kernal's `VideoScroll` currently moves 920 bytes through a RAM buffer, one 40-byte row at a time, 23 times, at roughly 30,000 cycles — about 30 ms at 1 MHz, nearly two frames. Hardware scrolling replaces it with one register write plus 40 stores to clear the newly exposed row: about 500 cycles. See §17.

Granularity is per pixel, in both axes, in every mode — including text. There is no cost to this: the renderer does the same arithmetic either way, and software that only wants whole-row scrolling simply writes multiples of 8. The Kernal should do exactly that, keeping a row origin and writing `row × 8`, and can then ignore that the hardware is finer than it needs. Smooth scrolling is there for anything that wants it — a credits roll, a menu, a message line — without the Kernal having to know.

Scroll values are sampled per display line, as §3 builds each one, so writing `LxSCRX` from a scanline interrupt bends the layer from two lines below `IRQLINE`. The scroll registers apply in the legacy submode too (§9).

SECTION 14

Interrupts

`/INT` is asserted while any latched source is still enabled, and released when acknowledged. It is level-driven and shares the AC6502 IRQ line, so a handler must chain to the Kernal's as described in the AC6502 interrupt documentation.

A frame begins at screen line 0 (§3): display line 0 in Graphics and Full, 238 in Text and Compact, where the 24 lines before the picture are top border.

SOURCE	IRQEN BIT	LATCHED IN STAT1	FIRES
Vertical blank	b0	b0	End of the active picture — the start of display line 192 in Text and Compact, 240 in Graphics and Full. Exactly once a frame
Scanline compare	b1	b1	Start of the line matching `IRQLINE`; `STAT2` reads `IRQLINE` in the handler. Every match, so a handler that reprograms `IRQLINE` gets another
Sprite overflow	b2	b2	As the frame's first line that drops a sprite is built (§3). At most once a frame
Sprite collision	b3	b3	As the frame's first colliding pixel is built. At most once a frame

`MODE1` b5 is an alias for `IRQEN` b0, so legacy code that enables the vblank interrupt through register 1 still works. `STAT0` b7 sets at the end of the picture regardless of `IRQEN`, so software can poll for vertical blank without enabling an interrupt at all (§6).

A source latches only while it is enabled. A source that fires while its `IRQEN` bit is clear leaves no trace in `STAT1`, so a handler reading it sees its own interrupts and nothing else. `STAT1` reads the latches masked by `IRQEN` as it is now, and `/INT` is asserted exactly while that is non-zero: clearing a source's `IRQEN` bit releases `/INT` without acknowledging it, and setting the bit again before anything is read asserts `/INT` again — as a TMS9918's `F AND IE` does. The flags in `STAT0` are not interrupts and do not follow this rule — b7, b6 and b5 set whether or not anything is enabled, which is what makes polling work (§6).

"At most once a frame" holds however quickly a handler acknowledges: an overflow acknowledged the moment it arrives is not raised again by the next overflowing line of the same frame. Overflow and collision belong to the frame whose line is being built, so in Graphics and Full the ones on display line 0 arrive as the frame before begins display line 261.

Vertical blank across a change of picture height. Precisely, vertical blank fires at the first line start of a frame whose screen line is at or past the end of the picture in the mode in effect as that line begins — screen line 216 in Text and Compact, 240 in Graphics and Full. Without a mode change that is display line 192 or 240. With one it is still exactly once: a change from 240 lines to 192 made after screen line 216 raises it as

the next line begins, and a change from 192 to 240 made after it has fired does not raise it again. Every geometry's picture has ended by screen line 240, so no frame goes without one.

A pending `FONT` load is carried out at that same line start, before F sets or `STAT1` b0 latches, so software that waits for either finds it complete (§7).

Acknowledge by reading `STAT1`, which clears every latch, or `STAT0`, which clears the vertical blank, overflow and collision latches with its flags (§6).

Timing on the hardware

Measured on the PRO (informative; §18 has the method):

- **Vertical blank** fires once a frame, every 262.5 display lines — 5,872,649 cycles at 352 MHz, 16.6837 ms, 59.94 Hz — with a spread of a few hundredths of a microsecond from one frame to the next.
- **Each scanline compare** fires where §3 puts its line, one display line (63.557 µs) after the line before, within a tenth of a microsecond, in every geometry and for every `IRQLINE` from 0 to 255. Across the odd VGA line the step is a line and a half: between display lines 237 and 238 in Text and Compact, and between 261 and 0 in Graphics and Full.
- **`/INT` falls about 1.4 µs after its line begins** (3.2 µs at most), and a line begins at the back porch before its first VGA line: so `/INT` falls near that line's first visible pixel. Status is staged with it (§6).
- **Overflow and collision** are published once the line that found them is built, which is before it is shown: 41 µs after the start of the line before it on average, 60 µs at most (§3).

The vertical blanking window

Vertical blank fires when the **picture** ends, not when the frame does, so the window between the flag and the next visible line is everything that is not active display — bottom border, blanking and top border together:

MODE	ACTIVE	WINDOW	TIME	CYCLES @ 1 MHZ	@ 2 MHZ
Text, Compact	192 lines	70 lines and the odd VGA line (§3)	4.48 ms	~4,480	~8,960
Graphics, Full	240 lines	22 lines and the odd VGA line	1.43 ms	~1,430	~2,860

The 192-line figure is the TMS9918's 70 lines and half a line more — which is the point. Software written against a real VDP gets the window it was written for.

The 240-line modes give up two thirds of it. That is the true cost of a full-height picture, and it is easy to miss: Graphics and Full have no borders to hide work in, so roughly 1,400 cycles is all the tear-free time there is. Filling a 960-byte name table takes about 8,600. Two ways out, both standard:

- **Update behind the raster.** Set `IRQLINE` partway down the picture and, in the handler, rewrite the rows the beam has already passed. The window becomes as large as the part of the screen you are willing to update late.
- **Accept the tear** where it does not show — a distant background layer, or a cell the player is not looking at.

Nothing here is new; it is the same arithmetic every full-screen 8-bit game has done. It is written down because the mode table makes 240 lines look free.

`IRQLINE` is eight bits and display lines run to 261, so lines 256–261 — the last six, in the top border in the 192-line modes and in blanking in the 240-line ones — cannot be selected. Nothing useful happens there.

Scanline interrupts are expensive on this CPU. One compare register means one interrupt per frame unless the handler reprograms `IRQLINE` on its way out. Entry and exit alone cost roughly 50 cycles before the handler does anything useful; at 1 MHz, an interrupt every eight lines costs about 10% of the machine on overhead alone. Reserve them for a handful of raster splits — a status bar, a palette change, a scroll seam — rather than a per-line effect.

SECTION 15

Reset State

After `RST`:

- All registers take the reset values in §5. `VMODE` = `$0`, so the legacy submode is in effect and `M1` / `M2` / `M3` select the mode — which, with `MODE0` = `MODE1` = `$00`, is Graphics I. Display **off**, interrupts disabled, layer 1 disabled, sprites enabled with `SPRCOUNT = 32`, `SPRLIMIT = 16` and the `$D0` terminator active. `L0CTRL` holds `$3C`, but the legacy submode overrides its depth, attribute source and opacity: layer 0 is Graphics I, colored per pattern group from a 32-byte table at `L0ATTR × $40` = `$0000`.
- VRAM contents are **undefined** except `$FC00`–`$FDFF`, which holds the default palette, and `$0800`–`$0FFF`, which holds the built-in font (§7), written after the palette. Software must not rely on the rest being zero.
- The palette cache is loaded from the default palette.
- Any pending `FONT` load is cancelled (§7).
- Both port pairs: pointer 0, direction read, prefetch byte 0, flip-flop cleared, `STATSEL` 0.
- The raster does not stop. The screen line being scanned carries on, the frame's vertical blank is not raised a second time, and the display line is numbered for the reset mode — a 192-line one — from the next line start (§3). Where the raster stands at power-on is undefined.

- `/INT` released; every `STAT0` flag, `STAT7`, the collision map and every `STAT1` latch clear.

The existing Kernal's `InitVideo` — eight register writes of `$00 $D0 $00 $00 $01 $00 $00 $1F` followed by a 2 KB character set upload — takes this to a working 40 × 24 black-on-white text screen with no changes. The upload still works, and writes the same bytes reset already put at `$0800`; on this card it is redundant.

SECTION 16

Detection

```
; Returns carry set if a 6502-PICOVDP is fitted
DetectVdp:
  lda VC_STATUS                 ; resynchronize the command flip-flop (§4)
  lda #$04                      ; select STAT4
  sta VC_REG
  lda #$8F                      ; register $0F | $80
  sta VC_REG
  lda VC_STATUS
  cmp #$AC
  beq @found
  clc
  rts
@found:
  lda #$00                      ; put STAT0 back
  sta VC_REG
  lda #$8F
  sta VC_REG
  sec
  rts
```

> On a real TMS9918 only three register bits are decoded, so `$8F` writes register 7 — the payload `$04` becomes transparent-on-dark-blue. The probe leaves the screen colors changed there: run it before `VideoSetColor`, or save and restore register 7 around it. Both status reads also clear the TMS9918's `F`, fifth-sprite and collision flags.
>
> A TMS9918 status byte can itself read `$AC` — `F` and `C` set, sprite field 12 — which is why the listing reads status once first: with the flags cleared, a false match needs a frame to end and two sprites to collide within the probe's few dozen cycles. Software that must never be fooled can confirm with `STAT6`. On this card that first read clears `STAT0`'s flags too, if `STATSEL_A` is 0 (§6).

`STAT5` gives the firmware version and `STAT6` the capability bits, for software that wants to degrade gracefully across future firmware.

A Kernal that relies on the built-in font (§7) instead of uploading its own checks `STAT6` b7 once the probe has found the card:

```
; After DetectVdp returns carry set: carry set if the card has the built-in font
DetectFont:
  lda #$06                      ; select STAT6
  sta VC_REG
  lda #$8F                      ; register $0F | $80
  sta VC_REG
  lda VC_STATUS
  asl                           ; b7 into carry
  lda #$00                      ; put STAT0 back; lda and sta leave carry alone
  sta VC_REG
  lda #$8F
  sta VC_REG
  rts
```

`STAT4` = `$AC` and `STAT6` b7 set are the whole test. `STAT5` is informational: don't require a value of it.

SECTION 17

BIOS Impact

The Kernal touches the VDP in 32 places, all in `Kernal.asm`. BASIC, the Monitor and Wozmon touch it nowhere.

Works unmodified

ROUTINE	WHY
`ProbeVideo`	Writes `$A5` to `$0000` and reads it back through the prefetch (§4)
`InitVideo`	`$D0` to register 1 selects text mode; registers 2 and 4 place the name and pattern tables where this VDP expects them
`InitCharacters`	Text patterns are still 1bpp, 8 bytes per character, 2 KB at `$0800`. It writes the bytes reset already put there (§7)
`VideoClear`	960-byte name table at `$0000`
`VideoSetCursor`, `VideoGetCursor`	Address arithmetic is unchanged: row × 40 + column
`VideoPutChar`, `VideoChroutRaw`	Command protocol unchanged
`VideoSetColor`	`COLOR` register semantics unchanged; the default palette makes the same nibble values the same colors
`VideoScroll`	Works, but see below

That is the whole text path. **No BIOS change is required to boot.**

Worth changing

`VideoScroll`. The current implementation costs ~30,000 cycles per line scrolled — 23 rows × (40 reads + 40 writes + address setup), about 30 ms at 1 MHz. Replacing it with a hardware scroll — guarded by §16's probe for as long as a Kernal must also run on a TMS9918, where register `$14` is register 4, the pattern table base:

```
; Scroll up one text line
  lda VID_SCROLL_Y
  clc
  adc #8
  cmp #192
  bcc @noWrap
  sbc #192
@noWrap:
  sta VID_SCROLL_Y
  sta VC_REG
  lda #($80 | $14)              ; register $14 = L0SCRY
  sta VC_REG
  ; then clear the row that just appeared at the bottom
```

About 500 cycles including the row clear and the address of the row to clear — roughly 60× faster. The cost is that the Kernal must track a scroll origin (`VID_SCROLL_Y` above is a new variable, not an existing one) and fold it into the row × 40 + column address calculation, which is one addition and a modulo-960 wrap.

The font. A Kernal written for this card drops its character set and `InitCharacters`, and relies on the font reset puts at `$0800` (§7), after checking `STAT6` b7 (§16). That frees 2 KB of ROM and a 2 KB upload at every boot: 8 ms at §4's ceiling, and two to three times that as a loop. Anything that overwrites the pattern table, such as a program going back to text from a graphics mode, restores it by writing `FONT` and waiting for vertical blank rather than by carrying a copy.

Port B for interrupt handlers. The Kernal brackets no VDP work with `sei` / `cli` today, and its own interrupt handler never touches the VDP; the hazard is a user handler installed through `IRQ_PTR` landing between the two writes of a command. On this card that handler uses port B (`$9C02` / `$9C03`), acknowledges through `STAT1`, and restores `VBANK` and `VINC` if it writes them (§4, §6). Worth adopting as a convention across the Kernal and in documentation — on this card only: on a TMS9918, `$9C02` / `$9C03` reach the same port as `$9C00` / `$9C01`.

New Kernal entry points to consider. Nothing here is required, but the following would make the new capabilities reachable from BASIC and from ordinary assembly without a programmer having to drive registers by hand: set mode, set a palette entry, set layer scroll, load a tile set, place a sprite, enable a layer. For text, `FONT` already loads the tile set.

Breaks

WHAT	EFFECT
Graphics II	Programs using it get Graphics I geometry and will draw garbage. `graphics-2.asm` (6502-DOCS, `samples/assembly/`) and the Graphics II part of the documentation's graphics chapter need rewriting.
Multicolor mode	Same — Graphics I geometry, wrong picture. `multicolor.asm` in the same place.
16 KB VRAM wrap	A pointer running off `$3FFF` now continues into `$4000` instead of wrapping to `$0000`.
Register decode	A register number above 7 no longer aliases onto 0–7. The F18A unlock sequence, and `f18a-detect.asm`, write VDP registers instead.
`$9C02`/`$9C03`	No longer mirrors of `$9C00`/`$9C01`: they are port B.
Sprites per line	16 by default, not 4: a program that relied on the fifth sprite vanishing, or on the fifth-sprite flag, sees neither until 17 cover a line — 33 if it raises `SPRLIMIT`.

Text mode and Graphics I keep working, so the BIOS boots untouched and `graphics-1.asm` (6502-DOCS, `samples/assembly/`) still runs. Both should gain a note that they are legacy modes and that Graphics and Full supersede them.

The natural replacement for `graphics-2.asm` is Graphics mode at 1bpp with the per-pattern-row attribute source — the same technique Graphics II used for its coloring, without the three-bank name indexing. It will not be a line-for-line port, because it no longer needs to be: 256 tiles with a color pair per row, or 512 at 4bpp with 16 colors each, is a different proposition from 768 monochrome ones.

Sprites keep TMS9918 semantics — one color, Y + 1, `$D0` terminator, early-clock bit, no sprites in Text (§9) — for as long as the program stays in the legacy submode, which it does by never writing `VMODE`.

The emulator's `src/core/IO/Video.ts` has had the same treatment the firmware needs, and is the reference implementation (§18).

SECTION 18

Implementation Notes

Firmware shape

The PICO9918 structure carries over, with both cores building the picture. Core 0 drives VGA timing: at the start of each screen line (every second VGA line) its sync program raises an interrupt, and core 0 signals core 1, whose interrupt is §3's latch. Core 1's renderer then evaluates the sprites and divides the next display line between the cores at a column chosen to balance them. Each core builds its half of the line — both layers and the sprites, by §12's priorities, into a 320-byte palette-index line of its own — and expands its columns through a 256-entry `uint32` lookup (one source pixel → two output pixels) into the RGB buffer the line is sent from. All of it happens within the

current display line, which is what §3's latch point gives the renderer. Bus accesses arrive as PIO interrupts on core 1, above everything else there: the latch holds them off only for its own moment — the line's events and where the writes before it end — and a row's status is published in a few dozen cycles, so no access waits long for either. Each access restages what the four ports will read next before the CPU's next access can arrive.

A `FONT` load (§7) is part of the line start where vertical blank fires: the render side takes it with that latch, and the bus side's 2 KB for each destination are copied behind it by core 1's renderer, a 64-byte chunk at a time, accesses served in between. An access that reaches a chunk not yet copied copies that chunk first, so every access after the line start finds the font whole. What that costs is measured below.

No pixel of a line depends on any other column's, so the halves are exact wherever the line is divided: priority among sprites and collision are resolved within each half's own pixels. Every status change — overflow, collision, their interrupts — is published by core 1, once both halves are built.

What changes:

- `vrEmuTms9918` is not used. The renderer is new.
- Core 0, which in the PICO9918 does nothing but VGA, builds half of every line.
- `tmsRead.pio` and its handler are rewritten: MODE1 is sampled alongside MODE, and a second port's prefetch and status byte have to be staged beside the first's, which the 32-bit word the read program works from has no room for (§2).
- `tmsWrite` needs no PIO change — MODE1 already arrives in bit 31 of the FIFO word. Only the handler changes.
- Two sets of pointer/prefetch/flip-flop state instead of one.
- VRAM grows from 16 KB to 64 KB. Comfortable in 520 KB.

Unpacking tables

A lookup from one pattern byte plus a group number to output bytes applies the palette group during unpacking:

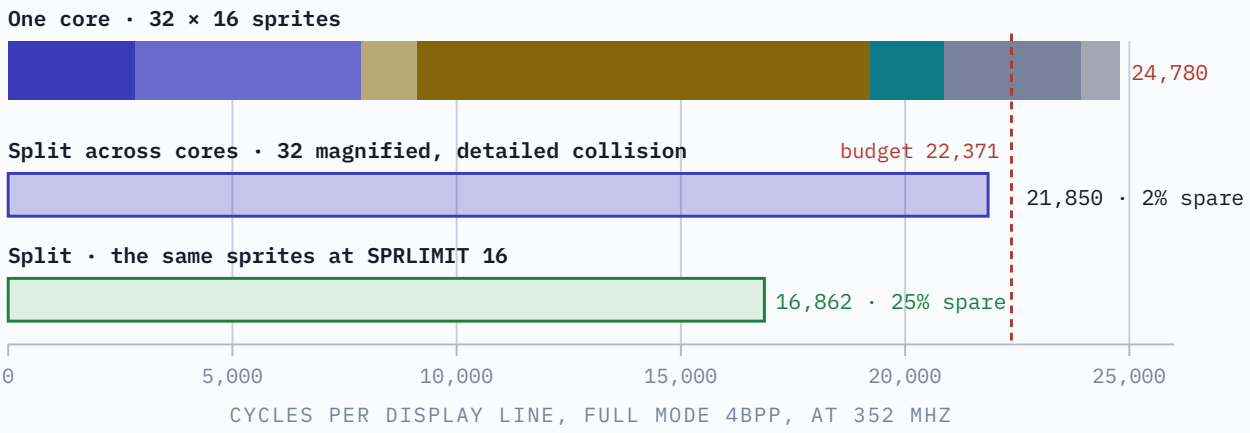
DEPTH	TABLE	SIZE	PER TILE ROW
1bpp	16 nibble-values → 4 bytes of mask, selecting fg or bg	64 B	two loads, two 32-bit stores
2bpp	256 byte-values → 4 values, the group added	1 KB	two loads, two 32-bit stores
4bpp	256 byte-values × 16 groups → 2 bytes	8 KB	four loads, two 32-bit stores
8bpp	none	—	two 32-bit loads, two stores

The 4bpp table is the one worth building. Measured against the best arithmetic unpacking, it saves 16% of an opaque 4bpp layer's cycles and 4% of a merged one's.

Measured per-scanline budget

A display line is two VGA lines, 63.56 µs, and the renderer has one display line to build the next: **22,371 cycles** at 352 MHz. These figures are measured on an RP2350 (a Raspberry Pi Pico 2, the PRO's silicon), with a renderer built to this design. The worst case measured is:

- both layers scrolled, with per-cell attributes, flips and priority
- 64 sprite slots covering the line
- every sprite pixel solid, and every sprite colliding with its neighbour



Layer 0, 320 px, 4bpp	2,846	Layer 1, 320 px, 4bpp with transparency merge	5,021
Sprite evaluation, 64 slots	1,258	Sprite composite, 32 × 16 px	10,087
Border and palette expansion, 320 px	1,662	Bus interrupt service, 32 accesses at 2 MHz	3,040
Line start: register snapshot, 32 VRAM writes	866	**Total, one core**	**24,780**
plus detailed collision (`SPRCTRL` b3), when enabled	*2,530*	*plus* magnified sprites	*3,185*

Measured cycles per display line, drawn to scale against the budget at 352 MHz. **One core cannot build the worst-case line in time; split across both cores, every measured worst case fits.** The split bars run from latch to expanded output with the bus and VGA interrupts running.

One core cannot build that line in time. That is why the sprites are built on core 0. Split that way, with the bus and VGA interrupts running for real, the longest line from latch to expanded output at 352 MHz is:

FULL MODE, 4BPP, TWO LAYERS	32 SPRITES ON THE LINE	SPARE	16 SPRITES ON THE LINE	SPARE
16 × 16 sprites	19,448	13%	16,475	26%
16 × 16, detailed collision	20,255	9%	16,501	26%
Magnified	20,962	6%	16,800	25%
Magnified, detailed collision	21,850	2%	16,862	25%

Every geometry and depth fits, and Full mode at 4bpp is the worst of them. `SPRLIMIT` resets to 16 (§5), so the reset state keeps a quarter of the worst line spare; software that wants 32 sprites a line sets it, and the worst case still fits. Without sprites, Full mode's two 4bpp layers take 13,435 cycles with bus service and line start: 40% of the line spare.

Bit depth moves the layers' cost less than it might seem. 8bpp is the cheapest layer to render and 4bpp the dearest, but only about 30% apart, because most of a cell's cost is fetching its name, attribute and pattern and merging it by priority. Sprite cost barely depends on depth. Depth costs VRAM and upload time more than render time.

The budget is measured at 352 MHz, and the firmware runs there (§2).

Measured with the firmware

The figures above are from the timing spike. The firmware, whose renderer implements this document and builds each line in two halves (Firmware shape, above), was measured on the same silicon over the same worst cases, at 352 MHz, with VGA's interrupts running and snapshots streaming over USB. The longest line from latch to expanded output, with no bus traffic:

FULL MODE, 4BPP, TWO LAYERS	32 SPRITES ON THE LINE	SPARE	16 SPRITES ON THE LINE	SPARE
16 × 16 sprites	17,009	24%	13,368	40%
16 × 16, detailed collision	18,921	15%	14,232	36%
Magnified	18,319	18%	14,305	36%
Magnified, detailed collision	20,164	10%	15,470	31%

No line was late in any geometry, depth or sprite setting, run for ten minutes between them, nor in the last row of the table, run ten minutes alone. Full mode at 4bpp is still the worst.

With the bus stand-in the spike's table assumed — a VRAM write every 2 µs, on the core that takes the bus — the same lines take:

FULL MODE, 4BPP, TWO LAYERS, BUS EVERY 2 MS	32 SPRITES ON THE LINE	SPARE	16 SPRITES ON THE LINE	SPARE
16 × 16 sprites	21,005	6%	16,990	24%
16 × 16, detailed collision	27,885	late	18,025	19%
Magnified	23,009	late	17,667	21%
Magnified, detailed collision	80,364	late	18,874	16%

So at the reset `SPRLIMIT` of 16 every case fits, with 16–24% of the line spare rather than a quarter. At 32 under that traffic, detailed collision or magnification makes lines late in Full mode at 2 and 4bpp, and both together do in Full mode at 1 and 8bpp and in Graphics mode at 2 and 4bpp. The real bus interface's cost, under a real CPU, is measured on the PRO.

A `FONT` load's copy follows the latch that carries it out (Firmware shape, above): for loads into both layers, 64 chunks of about 150 cycles each, done by core 1's renderer in the vertical blank's lines, which build nothing, and interrupted by the bus as it needs to. With loads for both layers every frame, all 36 measured scenes ran with no late line and no wrong row, and so did the last row of this section's first table, alone for ten minutes.

Measured on the PRO

The firmware on the PICO9918 PRO, driven through its pins by a bench harness that plays accesses at a 6502's spacing and answers its interrupts within a few microseconds, at 352 MHz:

FULL MODE, 4BPP, TWO LAYERS, 2 MHZ BURSTS	32 SPRITES ON THE LINE	SPARE	16 SPRITES ON THE LINE	SPARE
16 × 16 sprites	24,354	late	18,076	19%
16 × 16, detailed collision	30,121	late	19,350	14%
Magnified	28,612	late	19,471	13%
Magnified, detailed collision	33,447	late	20,935	6%

The bursts are 100 accesses exactly 2 µs apart — reads back to back, reads straight after their address command, writes — the tightest a 6502 at 2 MHz makes. The bus interrupt takes about 150 cycles an access in the debug build measured, half as much again as the stand-in above, so at `SPRLIMIT` 32 every case makes late lines under this traffic. At the reset `SPRLIMIT` of 16 every case fits, and no access was wrong in any of them.

The last row at 16, with `FONT` loads for both layers every frame, a scanline interrupt every eight lines served by the harness, and snapshots streaming over USB as well, ran for thirty minutes with the harness on each pair: no late line and no bus FIFO overrun, every snapshot right, all 33.6 million trials of accesses 2 µs apart right, and the worst line 3.6% inside its budget.

Late lines

A line is late if its build has not finished when its first VGA line begins. No line was late at the reset `SPRLIMIT` in any case measured above. But the worst of them has little spare, and traffic heavier than the measurement's is conceivable, so a late line is specified:

- It shows the most recently completed line again, for both of its VGA lines.
- Its build still runs to the end. Overflow and collision found in it reach `STAT0`, `STAT1`, `STAT7`, the collision map and `/INT` exactly as they would have (§6, §14).
- Nothing else is affected. The display line counter, the next line's latch (§3), VRAM, the palette and the registers all carry on as if the line had been on time.

The firmware counts late lines in its debug build.

Build order

1. Register model and VRAM in the emulator (`Video.ts`) — cheap to iterate. *Done: `6502-EMULATOR` 's 3.0.0 release, which implements the whole of this specification.*
2. Scanline renderer spike on the RP2350, timed, no bus interface. *Done: measured above, with the sprites on core 0.*
3. Bus interface: four ports, two pointer sets, PIO change.
4. Text mode via the legacy submode, and boot the unmodified BIOS.
5. `VMODE` , the remaining geometries, and the attribute sources — legacy Graphics I falls out here, and `graphics-1.asm` becomes the regression test.
6. Bit depths 2, 4 and 8; layer 1; scrolling.
7. Sprites.
8. Interrupts.
9. Kernal changes: hardware scroll, port B in the IRQ handler, new entry points.

Step 4 is the milestone worth reaching first: an unmodified BIOS booting to an `OK` prompt on a VDP with none of the TMS9918 left inside it.

The emulator as reference

`6502-EMULATOR` 's `src/core/IO/Video.ts` implements every section of this document, cites them by number, and is held to it by unit tests and by golden frames of real programs. Where firmware and emulator disagree, one of them is wrong, and it is settled in this document first. Two cartridges written for this card, in the emulator's `samples/` , make test programs: `vdp-modes` cycles the four geometries at 1, 2, 4 and 8bpp, and `vdp-layers` scrolls two 4bpp layers in Full mode past sprites at §12's levels 1 to 6. The emulator completes a `FONT` load at the same line start the firmware does (§7), so the two agree on when a load has landed.

It approximates the hardware in a few places, none of which a program should be able to rely on:

- **Frame timing.** 262 equal display lines at exactly 60 Hz, with no odd VGA line: its vertical blanking windows are 70 and 22 lines, about 0.6% and 2% short of §14's times. The golden frame numbers of the two cartridges assume this, and will not line up on hardware.
- `STAT3` **b1** is modelled as the last fifth of each of a display line's two VGA lines, measured from the start of the display line.
- `STAT5` reports the revision of this document, `$05` , having no firmware of its own.
- **A cold start** — a power cycle — zeroes VRAM before the palette is installed, and starts the raster at display line 0 of the reset mode, screen line 24. §15 leaves both undefined.
- It presents a frame to its host when the frame's last row has been built.
- It builds every line in time, so it never shows a late line (§18).

SECTION 19

Deliberate Omissions and Future Space

NO BLITTER

A fill/copy engine would be the single largest practical win for a 1 MHz CPU — clearing a 960-byte table would go from 3.8 ms to nothing — and it is nearly free to implement, being a `memset` on the Pico. It is left out to keep this revision a pure VDP. Registers `$28` – `$2F` are the natural home if it is added later; `STAT6` b6 is reserved to advertise it. The built-in font's register and capability bit, `$30` and `STAT6` b7, stay clear of both.

NO PER-SPRITE SIZE

The attribute byte is full. A fifth attribute byte, or a second sprite bank with a wider entry, would solve it.

NO MULTICOLOR MODE

Its 4 × 4 blocks come from the pattern table directly with no name indirection, which is a render path shared with nothing else. Graphics at 4bpp does everything it did and more.

NO GRAPHICS II

This one was close, and was in an earlier draft. It works by splitting the screen into thirds and giving each third its own 2 KB pattern bank, so that all 768 cells of a 32 × 24 screen can hold a unique 8 × 8 pattern — the TMS9918's way of faking a bitmap mode. Reproducing it needs a 10-bit pattern index that exists nowhere else in the design, `L0PAT` and `L0ATTR` read as bit flags rather than as base addresses, and the R3/R4 name-index masks, which are fiddly enough to get subtly wrong. The earlier draft skipped the masks and carried a caveat that a masked program "will render incorrectly" — and a mode that silently mis-renders in an undocumented corner is worth less than a mode that is honestly absent.

What it offered that the new engine does not is **768 unique monochrome cells**. What the new engine offers instead is 512 tiles with 16 colors each, which is a straight trade up for anything that was using Graphics II as a drawing surface. Its coloring scheme survives as attribute source `10` (§8), available at any geometry and any time.

If a true drawing surface is ever wanted, a linear bitmap mode is the better answer than Graphics II was: 1bpp over 256 × 240 is 7.5 KB and 31 ms to upload, 2bpp is 15 KB and 61 ms. Both are plausible on this CPU; 4bpp at 30 KB and 123 ms is not.

NO SPRITE-TO-LAYER COLLISION

The NES's sprite-0 hit exists to time raster splits; real scanline interrupts make it unnecessary here.

NO WIDER TEXT MODE

6 × 8 cells at 40 columns leave the frame 80 pixels wide of border. Full mode at 1bpp with per-cell attributes is the wider text screen this design has: 40 × 30 of 8 × 8, no border. What is not here is a 6-pixel cell across the whole 320 pixels — 53 columns — which would be a fifth geometry; `VMODE` `$5`–`$F` are free for it. In Full mode font `$00`'s glyphs sit in the left six pixels of each 8 × 8 cell; `FONT` IDs `$01`–`$7F` leave room for a true 8 × 8 set for that 40 × 30 console.

NO MAPS LARGER THAN THE SCREEN

Name tables the size of the screen — 768, 960 or 1200 bytes — keep every mode at 1 or 2 KB of alignment, at the cost of making scrolling a write-the-incoming-edge exercise. A larger map would be a `LxCTRL` bit selecting a 64 × 64 name table at 4 KB.

NO RASTER/DISPLAY LIST

Per-scanline register changes must come from the CPU through scanline interrupts, which are expensive here. A table of `(line, register, value)` triples executed by the VDP would make raster effects free.

APPENDIX

Resolved Design Questions

Recorded so the reasoning does not have to be reconstructed later.

TEXT MODE KEEPS ITS SECOND LAYER

Layer 1 in text mode costs render time that a single-layer text screen does not need, but it is the same code path as everywhere else and switching it off is one bit. A status bar or a frame held on one layer while the other scrolls is worth having available.

SCROLLING IS PER-PIXEL IN BOTH AXES, IN EVERY MODE

The renderer does the same arithmetic either way, so row granularity would be an artificial limit. The Kernal should write multiples of 8 and ignore that the hardware is finer (§13).

BOTH LIST TERMINATORS ARE KEPT

`SPRCOUNT` always bounds the sprite table — it makes evaluation a fixed cost and is what new software should use — and `SPRCTRL` b2 additionally re-enables the TMS9918's `$D0` terminator, which resets active. Software using the full 240-line height clears it, because `$D0` is row 208 and therefore on screen; legacy software never touches it, and the legacy submode forces it on regardless (§9). One bit, one comparison, no compromise either way.

The default palette is 16 rows of 16 (§11): the TMS9918 colors, a grayscale ramp, twelve hue ramps and two neutral ramps. Rows 2–15 are generated from a published formula rather than hand-picked, so the arrangement can be regenerated if the ramp

shape turns out wrong in practice.

THE DISPLAY LINE FOLLOWS THE RASTER

The VGA raster cannot move, so a change between a 192-line and a 240-line picture has to show up somewhere. Three places were weighed. Counting display lines from the top of the frame would remove the jump, but at the price of every raster split in the 192-line modes moving 24 lines under a mode change, which is what counting from the picture exists to prevent (§3). Deferring a change of height to the next frame would confine the jump to vertical blanking, but make that one kind of mode change take effect a frame late, unlike every other. So the jump happens where the change does, at the next line start, and §14 defines vertical blank by the screen line so that the frame it happens in still has exactly one. Software that changes height with the display off, during vertical blank — as mode changes are normally made — never sees it.

COLLISION DETECTION IS LAYERED

The TMS9918's single sticky bit stays, at no cost. Detailed per-sprite reporting — a 64-bit map across `STAT8` – `STAT15` — is available behind `SPRCTRL` b3, off at reset, because it is the one collision feature that costs real cycles (roughly 2,500 on a worst-case line, measured, for tracking which sprite owns each pixel). Anyone who never enables it pays nothing, and the compatible behavior is the default.

FULL MODE KEEPS TWO LAYERS AND 32 SPRITES A LINE; THE SPRITES MOVE TO CORE 0

Measured on the RP2350 (§18), one core cannot build Full mode's worst-case line in time at either clock preset. The remedies draft 0.3 listed were a 352 MHz clock, a lower default `SPRLIMIT`, and Full mode as a single-layer mode. They do not get there either, alone or together, without giving up features, and a single-layer Full mode would have broken software already written for two layers. Building the sprites on the second core does: every measured worst case fits at 352 MHz. `SPRLIMIT` still resets to 16, so the reset state keeps a quarter of the worst line spare as the spike measured it — 16–24% as the firmware measured it under the same bus stand-in, and 31–40% without (§18) — and a late line is specified for anything heavier than was measured.

THE 4BPP UNPACKING TABLE STAYS

It saves 16% of an opaque 4bpp layer's build and 4% of a merged one's against the best arithmetic unpacking. That is less than draft 0.3 assumed, but it costs 1.5% of the SRAM.

THE FONT LIVES IN THE CARD

Every Kernal for the AC6502 carries the same 2 KB character set and uploads it at boot. The card can hold it instead, for nothing (§7).

- **Why at reset, and at every reset.** Loaded at reset, like the palette, the font is simply there: a Kernal never uploads one, or commands one and waits, and a warm reset is no different from a cold one.

- **Why `$0800`.** It is where the Text layout (§7) and every existing Kernal put the pattern table. `L0PAT` resets to `$00`, which is the name table, so the address can't follow the register. A per-cell attribute table at `L0ATTR` = `$01`, `$0400` – `$07BF`, sits clear of it.
- **Why vertical blank.** A 2 KB copy is far more than the ~700 cycles an access may take (§4), so it can't happen as the write arrives. Done at the line start where vertical blank fires, it is part of work the card already does at a line start: every access after that line start finds it whole, it lands in blanking with no tear, and it lands on the same line in the emulator and the firmware, so the two can be compared exactly. The rule for software is the one it already knows: wait for F.
- **Why no status bit.** All sixteen status registers are taken and `STATSEL` is four bits, so a load has nowhere to report completion. Defining completion by time needs no bit: the F flag and the vertical blank interrupt already say when that time has passed.

APPENDIX

Still Open

Nothing structural. The three questions the last draft left open are resolved above: Graphics II is cut (§19), Full mode is in (§9), and 512 tiles per layer stay — which commits attribute bit 7 permanently and rules out using it for a second priority level or a per-tile depth override.

What remains is measurement, not design:

1. ~~**Time Full mode first.**~~ Resolved in draft 0.4: measured (§18), with the sprites moved to core 0 — see Resolved Design Questions.
2. ~~**Confirm the 4bpp unpacking table earns its 8 KB.**~~ Resolved in draft 0.4: it does, by less than assumed.
3. ~~**Check that the palette's hue ramps are usable in practice** rather than merely evenly spaced.~~ Resolved in Phase 10: the whole table was put on a PICO9918 PRO and captured off its DAC, every ramp still rising at every step on the monitor, and judged good as it stands. The formula (§11) stays as it is.
4. ~~**How fresh the status byte can be.**~~ Resolved in Phase 13: measured on the PRO, the byte is restaged within 3.2 µs of each line start, 1.35 µs on average, with `/INT`, and a read inside a line never showed another line's number. `STAT2`, `STAT3` b0 and the flags can be trusted to the line; b1 follows the horizontal blanking a few tenths of a microsecond late, and stays advisory (§6, §14).

Draft 0.2 closed the questions the emulator raised while implementing draft 0.1, draft 0.3 the ones planning the firmware against the VGA raster raised, draft 0.4 the first two above, on silicon, and the PRO the last two; they are listed under Revision History.

APPENDIX

Revision History

Draft 0.5

The card's own font. Normative changes:

- **Reset loads a built-in font** (§7, §15): CP437, 6 × 8, 2,048 bytes whose SHA-256 is normative, written to `$0800`–`$0FFF` after the default palette, at power-on and at every `RST`. VRAM there is defined after reset.
- **Register `$30` `FONT`** (§5, §7) loads it into layer 0's or layer 1's pattern table, and the load completes at the next vertical blank (§14). §5's reserved block is split around it.
- **`STAT6` b7** reports both (§6), so `STAT6` reads `$BF`. §16 shows the check.

The emulator reports `STAT5` = `$05` for this draft (§18).

No existing golden moves. Every golden fixture boots through BIOS 1.6, whose `InitCharacters` uploads the same 2,048 bytes to `$0800` before the first checkpoint, and no fixture reads that range first. All fifteen checkpoints replayed exactly into `Video.ts` with `$0800`–`$0FFF` seeded after a cold reset, once with the font and once with garbage. So the reset font needs a fixture of its own to prove it.

Amended when the firmware first loaded a font (informative; nothing normative changed): §18 records what a `FONT` load costs the latch that carries it out, about 9 µs with loads for both layers, and that a bus access arriving then waits for it; §4 points to the figure.

Amended when the PRO's raster was measured (Phase 13). Informative: §4, §6, §14 and §18 give the measured timing — the latch, the interrupts against the raster, how fresh status is, the bus under load at a 2 MHz CPU's pace, under which Full mode's sprite-heavy lines are late at `SPRLIMIT` 32 but not at the reset 16 — and Still Open 4 is closed. A `FONT` load's copy no longer holds accesses off: §4 and §18 say how it is done now. One sentence in §6 states what a read of a staged status byte clears — what it returned, and nothing that set after — which the emulator, which stages nothing, already does. No golden moves.

Draft 0.4

Measured on the RP2350, with a renderer built to this design. Normative changes:

- **`SPRLIMIT` resets to 16**, not 32 (§5, §15). Every other reset value is unchanged. A legacy program sees up to 16 sprites on a line by default (§9, §17).
- **A late line is specified** (§18). It shows the most recently completed line again; its overflow and collision, and everything else, are unaffected.

Also: §18's budget is measured rather than estimated, and the firmware builds sprites on core 0; the design is budgeted at 352 MHz (§2). Still Open 1 and 2 are resolved. Detailed collision costs roughly 2,500 cycles, not 500 (§10).

The emulator implements it and reports `STAT5` = `$04`. No index or VRAM golden moved; the structural goldens record the new `SPRLIMIT`.

Amended when the firmware first ran (informative; nothing normative changed): §2 names the PRO's MCU precisely, the RP2354A; §18 describes the firmware building each line in two halves, one a core, and records the budget it measured.

Draft 0.3

Settled while planning the firmware, against the VGA raster the hardware drives — which the emulator does not have, so checking draft 0.2 against it could not raise these. Normative changes:

- **Screen lines** (§3). The frame is 262 screen lines and one odd VGA line whatever the mode, and the display line is the screen line less the picture's top border. The odd VGA line follows screen line 261: display line 261 in Graphics and Full, but display line 237 in Text and Compact, where draft 0.2 put it after 261 — among the top border's visible rows.
- **A change between a 192-line and a 240-line geometry moves the display line numbering by 24** at the next line start (§3). Draft 0.2 said the counter was not re-based, which a raster that cannot move does not allow: the picture would have sat 24 rows out of place.
- **A frame begins at screen line 0, and vertical blank fires exactly once in it** (§14), defined by the screen line so that a change of picture height cannot skip it or raise it twice. Without a mode change nothing moves: the events fall on the same display lines as before.
- **Reset does not stop the raster** (§15). Draft 0.2 restarted the display line counter at 0, which firmware cannot do without restarting the monitor's sync. Where the raster stands at power-on is undefined; the emulator starts it at display line 0 (§18).

The emulator implements all of it and reports `STAT5` = `$03`. No golden frame moved.

Draft 0.2

Settled while implementing draft 0.1 in the emulator, and against the PICO9918 firmware source. Normative changes:

- **The legacy submode is TMS9918-exact** where draft 0.1 said so and did not specify how (§9): index 0 always transparent whatever `L0CTRL` b5 holds; sprite Y drawn at Y + 1 with `$E1`–`$FF` negative; `$D0` always terminates; attribute b4–b6 and `SPRPAL` ignored; b3:0 an index into palette row 0; no sprites in Text.

- **Status reads are split, and the flags are sticky** (§6, §14). A `STAT0` read clears its flags, `STAT7`, the collision map and the matching latches; a `STAT1` read clears the latches only. Nothing else but a reset clears the flags. Vertical blank, overflow and collision fire at most once a frame; `/INT` and `STAT1` are masked by `IRQEN` as it is now.
- `LxPAL` **is the palette row** of a 4bpp layer with attribute source "none" (§8).
- **A sprite's pattern index counts 8 × 8 patterns**, with no bits ignored; a 16 × 16 sprite at index N uses N to N + 3 (§10).
- **Timing** (§3, §14): 262 display lines numbered 0–261 and an odd VGA line; line N built from the state at the start of line N − 1; the scanline compare at the start of the matching line; the border taken per line.
- **Stated where draft 0.1 was silent:** the port protocol's prefetch, pointer and flip-flop rules and `VBANK` sampling (§4); what each out-of-range register value does (§5); `STAT3`'s bits, the 16 status registers and selector width (§6); table address wrap (§7); sources 01 and 10 at 2/4/8bpp, and flipped Text cells (§8); reserved `VMODE` values and what the legacy submode leaves live (§9); sprite coordinates relative to the picture, clipping, what counts toward the per-line limit and collision, and priority among sprites before the layers (§10); the palette's ignored nibble and normative table (§11); the scroll formula (§13); display off and mid-frame mode changes (§3); the full reset state (§15).

Corrections: the per-line budget was priced against one VGA line rather than two (§2, §18); core 0 and core 1 were the wrong way round (§18); the read-side PIO change is a rewrite, not one word (§2); the stock firmware's 252 MHz default (§2); 1bpp reaches 256 tiles, not 512 (§8); upload figures are a ceiling (§4); the §19 text-mode note predated Full mode; §16's probe resynchronizes the flip-flop; §17's scroll estimate and its missing breaks.

6502-PICOVDP · draft 0.5 · implemented in the emulator and in firmware, measured on the PRO

AC6502 family · PICO9918 PRO v2.0